



Applied Computer Security to Industrial Environments

**Design and implementation of a DevSecOps
architecture based on GitOps:
Supply chain security in Kubernetes using
scanners, ArgoCD and Kyverno**

Master's Thesis

**Ziad El Karrabi El Hanafi
Supervisor: Josep Andreu**

2026

Contents

List of Figures	V
List of Tables	1
1 Introduction	3
2 Theoretical Framework and Reference Analysis	5
2.1 Evolution from the DevOps to the DevSecOps paradigm	5
2.1.1 Limitations of traditional models	5
2.1.2 Integration of security in the software lifecycle	5
2.1.3 Birth of the DevSecOps concept	5
2.2 Kubernetes as an execution platform	6
2.2.1 Kubernetes architecture	6
2.2.2 Main cluster components	6
2.2.3 Network management and Network Policies	6
2.3 Policy-as-Code and Admission Controllers	7
2.3.1 How Admission Controllers work	7
2.3.2 Resource validation vs. mutation	7
2.3.3 Importance of declarative security	7
2.4 Tools and technologies used	8
2.4.1 Kyverno	8
2.4.2 OPA Gatekeeper	8
2.4.3 ArgoCD and GitOps	8
2.4.4 GitHub Actions	8
2.4.5 Trivy and Sysdig Secure	9
2.5 Theoretical comparison: Kyverno vs OPA	9
3 Objectives and Scope	11
3.1 General objective	11
3.2 Specific objectives	11
3.3 Project scope	12
4 Project Methodology	13
4.1 General project approach	13
4.2 Laboratory environment	13
4.3 Project phases	14
4.3.1 Phase 1: Kubernetes cluster provisioning	14
4.3.2 Phase 2: Kyverno deployment and integration	14
4.3.3 Phase 3: Security policy implementation	15
4.3.4 Phase 4: GitOps integration with ArgoCD	15
4.3.5 Phase 5: CI/CD integration with GitHub Actions and Trivy	15
4.4 Selection and scope of security policies	15
4.5 System reporting and observability	16
4.6 Validation strategy	16

4.7	Comparison with OPA Gatekeeper	17
4.8	Scope decisions and anticipated limitations	17
5	Architecture Design	19
5.1	General system architecture	19
5.2	Component description	20
5.2.1	Kubernetes cluster	20
5.2.2	Kyverno	21
5.2.3	ArgoCD	21
5.2.4	GitHub Actions and Trivy	22
5.2.5	Sysdig Secure	22
5.3	DevSecOps deployment flow	22
5.3.1	Continuous integration phase	23
5.3.2	Continuous deployment and admission phase	23
5.4	Cluster network architecture	24
5.5	Design decisions	26
5.5.1	Separation between CI and CD	26
5.5.2	ClusterPolicy scope vs. Policy	27
5.5.3	Enforce mode vs. Audit mode	27
5.5.4	Mutation before validation as a hardening strategy	27
5.5.5	synchronize: true in generation policies	27
5.5.6	Pipeline supply chain security	28
5.5.7	Choice of Calico as CNI plugin	28
6	Cluster Implementation and Security Policies	29
6.1	Kubernetes cluster provisioning	29
6.2	Kyverno deployment and integration	30
6.2.1	Kyverno's autogen mechanism	31
6.3	Validation policies	34
6.3.1	Policy P1: disallow-root	34
6.3.2	Policy P2: disallow-privilege-escalation	35
6.3.3	Policy P3: restrict-image-registries	36
6.4	Mutation policies	38
6.4.1	Policy P4: add-default-resources	39
6.4.2	Policy P5: add-default-securitycontext	40
6.4.3	Policy P6: add-readonly-rootfs	42
6.4.4	Policy P7: add-default-labels	43
6.5	Generation policy	44
6.5.1	Policy P8: generate-networkpolicy-metadata	44
6.6	Operational considerations and lessons learned	47
6.6.1	Exclusion of infrastructure namespaces	47
6.6.2	Integral auto-hardening test	48
6.6.3	The mutation-validation order verified empirically	51
6.7	Policy Reports: compliance auditing	51

7	GitOps Integration with ArgoCD	53
7.1	GitOps repository design	53
7.2	ArgoCD deployment	55
7.2.1	Incident during installation: annotation size limit on CRDs	55
7.2.2	Deployment verification	56
7.3	Application for Kyverno policies	57
7.3.1	Deployment and synchronisation process	59
7.4	Managing false drift from Kyverno defaults	61
7.4.1	Evaluation of resolution options	61
7.5	Drift protection tests	62
7.5.1	Self-healing upon manual modification	62
7.5.2	Policy deployment via Git commit	63
7.5.3	Protection against deletion	65
7.6	Application for the demo workload	66
7.6.1	Demo application design	66
7.6.2	Application manifest and deployment	67
7.6.3	Verification of Kyverno mutations	69
7.7	Analysis: interaction between mutating webhooks and ArgoCD	71
7.7.1	Observed behaviour: absence of drift in the demo Application	71
7.7.2	Verification of the case where drift does occur	73
7.7.3	Conclusion and resolution pattern	75
	Bibliography	77

List of Figures

1.1	General architecture of the DevSecOps environment	4
5.1	General architecture of the DevSecOps environment	20
5.2	Continuous integration and deployment flow with security analysis	23
5.3	Kyverno admission flow: mutation before validation	24
5.4	Network architecture of the laboratory cluster	25
6.1	Kyverno controllers deployed in the cluster	30
6.2	ValidatingWebhookConfigurations registered by Kyverno in the API Server	31
6.3	Initial cluster state: no policies deployed	31
6.4	P1: pod without <code>runAsNonRoot</code> blocked by Kyverno's webhook	35
6.5	P2: pod with <code>allowPrivilegeEscalation: true</code> blocked	36
6.6	P3: simultaneous blocking by <code>disallow-root</code> and <code>restrict-image-registries</code>	38
6.7	P4: automatic injection of <code>resources.requests</code> and <code>resources.limits</code>	40
6.8	P5: automatic injection of <code>allowPrivilegeEscalation</code> and <code>capabilities.drop</code>	42
6.9	P6: verification of read-only filesystem at runtime	43
6.10	P7: traceability labels automatically injected in pods	44
6.11	P8: NetworkPolicy automatically generated upon namespace creation . . .	46
6.12	P8: <i>drift protection</i> — NetworkPolicy regenerated 57 s after manual deletion	47
6.13	All eight ClusterPolicies in Ready state after complete deployment	48
6.14	Minimal pod manifest used in the integral test	48
6.15	Integral test (part 1): pod metadata and resources after admission	49
6.16	Integral test (part 2): complete security profile injected by Kyverno	50
6.17	ClusterPolicyReport: evaluation of the generation policy on the test namespace	51
6.18	PolicyReports of the default namespace: 7 PASS per pod, 0 violations . .	52
7.1	Structure of the <code>tfm-gitops</code> repository with separation by functional planes	54
7.2	<code>kustomization.yaml</code> file in the <code>policies/</code> folder with all eight policies .	55
7.3	ArgoCD's seven components in Running state in the <code>argocd</code> namespace . .	56
7.4	ArgoCD CRDs registered in the cluster: <code>applications</code> , <code>applicationsets</code> and <code>appprojects</code>	56
7.5	ArgoCD v3.3.8 interface accessible at <code>https://192.168.1.101:8080</code> . .	57
7.6	Manifest <code>app-policies.yaml</code> : Application managing Kyverno's Cluster-Policies	58
7.7	State transition <code>OutOfSync</code> → <code>Synced</code> after first deployment of <code>kyverno-policies</code>	59
7.8	The eight ClusterPolicies restored by ArgoCD in Ready state	59
7.9	ArgoCD <i>ownership</i> annotation in the <code>disallow-root</code> ClusterPolicy	60
7.10	Tree view in ArgoCD: eight ClusterPolicies synchronised from the Git repository	60
7.11	End-to-end verification: insecure pod blocked by ArgoCD-managed policies	61
7.12	Test 1: manual modification reverted by ArgoCD via <code>selfHeal</code>	63
7.13	Test 2: new policy created via Git commit	64
7.14	Test 2: <code>disallow-latest-tag</code> automatically deployed by ArgoCD (2m42s age)	64
7.15	Test 2: ArgoCD tree updated with the new <code>disallow-latest-tag</code> policy .	65

7.16	Test 3: disallow-root policy restored by ArgoCD 48 s after manual deletion	65
7.17	Demo application deployment.yaml manifest with three emptyDir volumes	67
7.18	Manifest app-demo.yaml: Application for the workload with selfHeal: false	68
7.19	Resource tree of the demo-app Application: Namespace, Service, Deployment and two Pods	69
7.20	Verification of mutations P4, P7 and P8 on the demo application pods	69
7.21	Demo application accessible and functional at 192.168.1.101:8888 with all policies active	70
7.22	Final ArgoCD panel state: two Applications in Healthy and Synced state	71
7.23	Both Applications in Synced and Healthy state: pod mutation does not generate drift in ArgoCD	72
7.24	Deployment vs. Pod comparison: mutations exist in the Pod but not in the spec.template of the Deployment	72
7.25	Autogen rules for policy P4: cover Deployment, StatefulSet, Job, ReplicaSet via spec.template.spec	73
7.26	Experimental policy that directly mutates Deployment type resources	74
7.27	Real drift case test: demo-app transitions to OutOfSync when directly mutating the Deployment	74

List of Tables

2.1 Comparison between Kyverno and OPA Gatekeeper 9

1 Introduction

The adoption of container-based architectures and orchestrators such as Kubernetes has transformed how applications are developed and deployed in cloud and enterprise environments. This paradigm enables automated lifecycle management of applications, improves scalability and facilitates the integration of modern development practices. However, this technological evolution also introduces new security challenges, especially in environments where deployments are carried out in an automated and continuous manner.

In this context, the DevSecOps approach has emerged, with the objective of integrating security within the software development lifecycle itself, rather than treating it as a post-deployment phase. This model promotes the automation of security controls within continuous integration and deployment pipelines, enabling the detection of vulnerabilities, the application of security policies and the guarantee of secure configurations from the earliest stages of development.

This Master's Thesis focuses on the implementation of a complete DevSecOps environment on Kubernetes, integrating Continuous Integration tools (GitHub Actions) with automated vulnerability analysis using tools such as Trivy and Sysdig Secure. A GitOps-based Continuous Deployment model using ArgoCD is also implemented, enabling the declarative management of infrastructure and deployments through Git repositories.

Additionally, Kyverno is incorporated as a Policy-as-Code security policy engine, enabling the automatic enforcement of controls over Kubernetes manifests during the deployment process.

The core of the project is based on the use of Admission Controllers, specifically Kyverno, to apply security policies declaratively within the Kubernetes cluster. Unlike traditional approaches that only block insecure configurations, this work explores the use of mutating policies, capable of automatically correcting insecure configurations in YAML manifests at deployment time.

Finally, the project includes a comparative analysis between Kyverno and OPA Gatekeeper, evaluating their capabilities in implementing security policies in Kubernetes, their ease of use, learning curve and effectiveness in production environments.

Figure 1.1 shows a general overview of the DevSecOps architecture implemented in this work, where development processes, security analysis, continuous deployment and policy enforcement are all integrated within a Kubernetes environment.

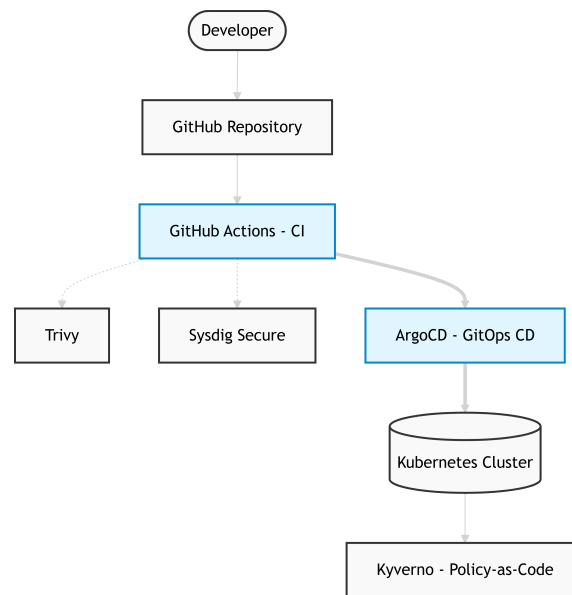


Figure 1.1: General architecture of the DevSecOps environment

2 Theoretical Framework and Reference Analysis

2.1 Evolution from the DevOps to the DevSecOps paradigm

2.1.1 Limitations of traditional models

Traditional software development models, such as the waterfall model, are characterised by a strict separation of phases (analysis, development, testing and deployment), which hinders the early detection of errors and vulnerabilities [1]. In these approaches, security is usually introduced in the final stages, significantly increasing the cost of correction and the risk in production.

The adoption of agile methodologies and, subsequently, the DevOps paradigm improved collaboration between teams and accelerated delivery cycles [2]. However, various studies highlight that security continued to be treated as an independent process, generating vulnerabilities especially in cloud-native environments [3].

In this context, the need to integrate automated security mechanisms that can be applied continuously within the software lifecycle becomes evident.

2.1.2 Integration of security in the software lifecycle

The evolution towards DevSecOps implies integrating security from the earliest phases of development, incorporating automated controls in CI/CD pipelines [4]. This includes vulnerability analysis, configuration validation and compliance control.

Automation enables consistency and reduces human errors, especially in dynamic environments. According to [5], the early integration of security significantly improves software resilience.

This approach is key in the present work, where security is implemented through automated mechanisms integrated in Kubernetes.

2.1.3 Birth of the DevSecOps concept

DevSecOps emerged as a natural evolution of DevOps, integrating security as an intrinsic component of the software lifecycle [6]. This paradigm promotes the automation of security controls and shared responsibility across teams.

In container and Kubernetes-based environments, this approach requires mechanisms capable of applying security policies dynamically and at scale.

In this work, this objective is materialised through the use of Policy-as-Code and tools such as Kyverno, which enable declarative and automated security enforcement.

2.2 Kubernetes as an execution platform

2.2.1 Kubernetes architecture

Kubernetes has consolidated itself as the de facto standard for container orchestration, providing a robust platform for the management of distributed applications [7].

Its architecture is based on a control plane, which manages the state of the system, and worker nodes responsible for running workloads. The API Server acts as the central interaction point through a declarative model [8].

This architecture enables automated system management, but also introduces new security challenges that must be addressed through specific mechanisms.

2.2.2 Main cluster components

The main control plane components include the API Server, the Scheduler and the Controller Manager, responsible for processing requests and maintaining the desired state of the system. On worker nodes, the kubelet and the container runtime enable the execution of pods [8].

The existence of multiple distributed components makes it necessary to implement centralised security controls, such as Admission Controllers.

2.2.3 Network management and Network Policies

By default, Kubernetes allows free communication between pods, which can represent a significant risk. Network Policies allow defining communication rules based on labels and namespaces, applying the principle of least privilege [8].

Furthermore, in real environments it is common to find insecure configurations such as:

- Containers running as the root user
- Privilege escalation enabled
- Missing resource limits

- Unrestricted network access

These practices pose a high risk, especially in *multi-tenant* environments —where multiple teams, applications or clients share a single Kubernetes cluster with logical isolation via namespaces— since a compromise in one workload can propagate to others if there are no adequate network, execution and privilege controls.

This justifies the need to apply automatic controls at deployment time through Kubernetes security policies.

2.3 Policy-as-Code and Admission Controllers

2.3.1 How Admission Controllers work

Admission Controllers are API Server mechanisms that intercept requests before resources are deployed in the cluster [8]. They allow applying validations or modifications to objects.

This mechanism constitutes the technical basis on which security is implemented in this work.

2.3.2 Resource validation vs. mutation

Validating admission controllers allow verifying that resources comply with certain policies, rejecting those that do not satisfy them.

On the other hand, *mutating admission controllers* allow automatically modifying resources before their creation, applying secure default configurations.

In this work, special emphasis is placed on *mutating* policies, as they enable automatic hardening mechanisms to be applied to manifests, correcting insecure configurations without user intervention.

This approach is key to ensuring security in highly automated DevSecOps environments.

2.3.3 Importance of declarative security

The Policy-as-Code approach allows defining security policies as versioned code, facilitating their integration into GitOps workflows [9].

This model improves traceability, reproducibility and automation of security.

In this work, these mechanisms are implemented using Kyverno to automatically apply policies

at deployment time.

2.4 Tools and technologies used

2.4.1 Kyverno

Kyverno is a Policy-as-Code tool designed specifically for Kubernetes that allows defining policies using YAML, without the need to learn additional languages [10].

Unlike other solutions, Kyverno allows defining policies directly in the same format used by Kubernetes, which facilitates its adoption. Furthermore, it supports both validation and mutation policies, enabling not only the blocking of insecure configurations but also their automatic correction.

This capability is especially relevant in DevSecOps environments, where automation and consistency are key factors.

Kyverno constitutes the central axis of this work.

2.4.2 OPA Gatekeeper

Open Policy Agent (OPA) is a general-purpose policy engine that allows defining rules using the Rego language [11]. Gatekeeper facilitates its integration with Kubernetes.

Although it offers great flexibility, its complexity and validation-focused approach differentiate it from Kyverno.

2.4.3 ArgoCD and GitOps

ArgoCD implements the GitOps paradigm, where the state of the system is defined in Git repositories [12].

In this work, it is used to automate deployment and ensure consistency between configuration and real state.

2.4.4 GitHub Actions

GitHub Actions enables the automation of CI/CD pipelines, including build, test and security processes [13].

It is used to integrate security controls within the DevSecOps workflow.

2.4.5 Trivy and Sysdig Secure

Trivy enables vulnerability scanning of container images [14], while Sysdig Secure provides runtime security [15].

These tools complement the security applied through Kubernetes policies.

2.5 Theoretical comparison: Kyverno vs OPA

Kyverno and Open Policy Agent (OPA) represent two of the most relevant solutions within the *Policy-as-Code* approach applied to Kubernetes. Both allow defining and applying security policies over cluster resources, although they present significant differences in terms of design, approach and capabilities [10], [11].

Kyverno is a *Kubernetes-native* tool that allows defining policies using the same YAML format used in the system's own manifests. This approach reduces complexity and facilitates adoption by DevOps teams familiar with Kubernetes. By contrast, OPA uses the Rego language, a general-purpose declarative language that offers great flexibility but introduces a steeper learning curve.

From a functional standpoint, both tools support the implementation of validation policies (*validating*), that is, checking that resources meet certain conditions before being deployed. However, a key difference lies in the support for mutation policies (*mutating*). Kyverno allows automatically modifying resources at admission time, applying secure default configurations, while OPA Gatekeeper focuses primarily on validation, lacking native support for resource mutation.

The following table presents a summarised comparison of both tools:

Feature	Kyverno	OPA Gatekeeper
Approach	Kubernetes-native	General-purpose
Policy language	YAML	Rego
Learning curve	Low	High
Resource validation	Yes	Yes
Resource mutation	Yes	No (limited)
Kubernetes integration	Native	Via CRDs
Ease of adoption	High	Medium

Table 2.1: Comparison between Kyverno and OPA Gatekeeper

These differences have a direct impact on their applicability in DevSecOps environments.

While OPA offers greater expressiveness and flexibility for defining complex policies, Kyverno stands out for its operational simplicity and its ability to integrate naturally into Kubernetes-based workflows.

From the perspective of this work, Kyverno's mutation capability is especially relevant, as it allows implementing automatic *hardening* mechanisms over manifests, correcting insecure configurations without the need for manual intervention. This includes, for example, the automatic addition of resource limits, the disabling of elevated privileges or the application of default network policies.

Consequently, Kyverno is positioned as a tool especially suited for environments where automation, consistency and integration with GitOps practices are prioritised.

This theoretical comparison will serve as the basis for the practical analysis developed in later chapters, where the behaviour of both solutions will be evaluated in a real Kubernetes environment.

3 Objectives and Scope

3.1 General objective

The main objective of this Master's Thesis is the design and implementation of a functional DevSecOps environment on Kubernetes, integrating security mechanisms natively within the software development and deployment lifecycle.

The core of the project focuses on applying the Policy-as-Code paradigm through the use of Admission Controllers, specifically Kyverno, with the objective of guaranteeing secure configurations at deployment time. Unlike traditional approaches based solely on detection or blocking, this work places special emphasis on the use of *mutating* policies, capable of automatically correcting insecure configurations in Kubernetes manifests.

This approach enables not only the prevention of configuration errors, but also the establishment of automatic hardening mechanisms aligned with modern security practices in cloud-native environments.

3.2 Specific objectives

To achieve the general objective, the following specific objectives are defined:

- Deploy a functional Kubernetes cluster in a laboratory environment, with an architecture based on one control node and one worker node, properly configuring communication and the network layer via a CNI solution such as Calico.
- Install and configure Kyverno as a Policy-as-Code security policy engine, integrating it as an Admission Controller within the cluster.
- Define and implement a representative set of security policies (between 5 and 10), focused on real-world issues such as:
 - containers running with elevated privileges,
 - privilege escalation,
 - absence of resource limits,
 - unrestricted network communication.

- Validate policy behaviour through practical tests, demonstrating both the blocking of insecure configurations (*validating policies*) and their automatic correction (*mutating policies*).
- Implement an advanced policy based on the automatic generation of Network Policies, with the objective of restricting access to sensitive resources, such as metadata services (for example, IP 169.254.169.254), common in cloud environments.
- Integrate the cluster within a GitOps-based continuous deployment model, using ArgoCD to manage the declarative state of applications.
- Incorporate a continuous integration pipeline via GitHub Actions, including build and vulnerability analysis processes with tools such as Trivy or Sysdig Secure.
- Perform a technical comparison between Kyverno and OPA Gatekeeper, analysing differences in ease of use, validation and mutation capabilities, and suitability in production environments.

3.3 Project scope

The project is developed in a laboratory environment based on virtual machines, with the objective of reproducing a realistic Kubernetes deployment scenario. The scope includes the cluster configuration, the integration of the main DevSecOps pipeline tools and the implementation of representative security policies.

However, the work does not aim to cover all existing security standards exhaustively, but rather to demonstrate the functionality and viability of the Policy-as-Code model applied to Kubernetes through practical case studies.

Additionally, the use of automation tools such as Ansible is considered a possible extension of the project. Following an iterative approach, priority is initially given to the implementation of security mechanisms and their validation, leaving the complete deployment automation as future or parallel work depending on available time.

4 Project Methodology

This chapter describes the methodological approach adopted to carry out the Master's Thesis. This is not a technical chapter, but rather a reasoned description of how the work has been planned, what decisions have been made regarding scope and project organisation, and what criteria have guided tool selection and the definition of development phases. Many of these decisions were agreed upon with the supervisor, Josep Andreu, through various meetings and communications that have progressively refined the approach of the work.

4.1 General project approach

The project adopts an approach of **applied research with practical development**. The objective is not to carry out an exhaustive theoretical review of Kubernetes security standards, but to build a real functional environment that demonstrates how to integrate security in an automated manner within the development and deployment lifecycle. Results are validated empirically through controlled tests on the implemented environment, not through bibliographic analysis.

The supervisor oriented the work towards a manageable number of between five and ten well-chosen security policies, with the argument that demonstrating the functioning and deep understanding of Kyverno is academically more valuable than applying a complete standard superficially. This guideline has shaped the entire project planning: quality and justification of each decision are prioritised over the quantity of elements implemented.

Development follows an **iterative and incremental** model: starting with the most fundamental components (the cluster and Kyverno), validating each element before adding the next, and progressively advancing towards components of greater complexity (ArgoCD, GitHub Actions, comparison with OPA). This model enables early detection of integration problems and guarantees that the environment is functional at all times, even before it is complete.

4.2 Laboratory environment

The environment on which the project is developed consists of a Kubernetes cluster deployed on two virtual machines provided by the supervisor, managed with VirtualBox. This decision was agreed upon during the initial exchange with Josep Andreu, who recommended using the virtual machine images used in the course subject to guarantee coherence and stability.

The two virtual machines have the following roles and fixed IP addresses within the laboratory network:

- **Control node** (*control plane*): 192.168.1.101, hostname `control`. Hosts the Kubernetes control plane and acts as the entry point for all cluster administration operations.
- **Worker node**: 192.168.1.102, hostname `worker`. Executes the workloads deployed in the cluster.

Each virtual machine is configured with two network interfaces: one NAT type for internet access, and one *Host-only Adapter* type for internal communication between nodes. The cluster network layer is implemented using **Calico v3.28** [16], a CNI plugin that allows defining and effectively applying Network Policies, which is a requirement for one of the security policies implemented in the project.

The choice of a local virtualised environment, rather than a cloud provider, responds to criteria of control and reproducibility. However, this decision has implications for the project scope, which are detailed at the end of this chapter.

4.3 Project phases

The project is organised into five sequential phases. Each phase has a defined scope and the next phase is not started until the previous one has been verified as functional.

4.3.1 Phase 1: Kubernetes cluster provisioning

The first phase encompasses the setup of the Kubernetes cluster on the two laboratory virtual machines. It includes the initialisation of the control node, the joining of the worker node and the installation of the Calico network plugin. The expected result is an operational cluster with both nodes active and all system components running.

4.3.2 Phase 2: Kyverno deployment and integration

The second phase introduces Kyverno as the security policy engine within the cluster. Kyverno is installed as a set of controllers that integrate with Kubernetes' Admission Controllers mechanism, intercepting requests to the API Server to apply the defined policies. This phase also includes the verification that Kyverno's webhooks are correctly registered and operational before proceeding with policy implementation.

4.3.3 Phase 3: Security policy implementation

The third phase constitutes the core of the work. A set of security policies covering the three types of actions available in Kyverno is designed and implemented: validation, mutation and generation. Policies are applied incrementally, starting with the simpler ones (validation of insecure configurations) and progressing towards the more complex ones (automatic manifest mutation and generation of derived resources).

4.3.4 Phase 4: GitOps integration with ArgoCD

Once the policies have been validated and are operational, the fourth phase introduces ArgoCD to manage the cluster state declaratively through a Git repository. All configurations — Kyverno policies, test manifests and infrastructure configurations — are stored in Git, and ArgoCD guarantees that the cluster state remains synchronised with the repository. This phase demonstrates the complete GitOps cycle: any change in the repository is automatically propagated to the cluster, and any manual deviation is detected and reverted.

4.3.5 Phase 5: CI/CD integration with GitHub Actions and Trivy

The fifth phase incorporates the continuous integration dimension into the DevSecOps environment. A pipeline is configured in GitHub Actions that automatically executes a vulnerability analysis on the container images used, employing Trivy as the scanning engine. This phase closes the complete DevSecOps cycle and additionally introduces the consideration of the risks of the pipeline's own supply chain, motivated by the supervisor's alert about the TeamPCP group attack, which compromised CI/CD pipelines through the manipulation of third-party actions and runners [17].

4.4 Selection and scope of security policies

The set of policies implemented does not aim to exhaustively cover all existing security controls for Kubernetes. Following the supervisor's guidance, between five and ten policies have been selected that illustrate the different action mechanisms of Kyverno and correspond to real, well-documented security problems.

The selected policies address the following risk categories:

- Execution of containers as the root user
- Privilege escalation at runtime

- Use of images from unauthorised registries
- Absence of resource limits (CPU and memory)
- Insecure container security context configuration
- Write access to the container filesystem
- Access to the cloud instance metadata service (IP 169.254.169.254)

The last policy — blocking access to the cloud metadata service — was explicitly proposed by the supervisor as an illustrative example of Kyverno's generation mechanism. Through a *generate* policy, Kyverno automatically creates a `NetworkPolicy` in each new cluster namespace that prevents access to that IP. If the `NetworkPolicy` is manually deleted, Kyverno regenerates it automatically, demonstrating the concept of protection against configuration deviations (*drift protection*).

Additionally, the supervisor pointed out the interest in incorporating image validation through digital signatures with tools such as Cosign or Notary, as well as the use of Kyverno's `imageRegistry` context to inspect image metadata. These functionalities are considered as a possible extension of the work if the available time permits, but do not form part of the committed scope.

4.5 System reporting and observability

The supervisor explicitly indicated that reporting is a relevant added value for the project. Kyverno automatically generates `PolicyReport` and `ClusterPolicyReport` resources that consolidate the results of policy evaluation over all cluster resources. These reports enable centrally knowing which resources comply with each policy, which have been automatically mutated and which have been rejected.

The review and documentation of these reports forms part of the system validation plan and will be included as evidence in the results chapter. This mechanism provides an audit and compliance dimension that complements individual functional tests and demonstrates the applicability of the system in real environments where traceability of security evaluations is a requirement.

4.6 Validation strategy

System validation is carried out through controlled tests on the laboratory environment. For each policy implemented, at least one test scenario is executed that demonstrates its behaviour

with direct documentary evidence: command outputs, system states and environment captures.

Tests are organised into three categories:

- **Blocking tests:** an attempt is made to deploy a resource that violates a validation policy and it is verified that the API Server rejects it with the corresponding error message.
- **Mutation tests:** a resource with an incomplete or minimal configuration is deployed and it is verified that Kyverno has automatically applied the expected security fields, without the developer having had to specify them.
- **Generation tests:** a namespace is created and both the automatic generation of the associated NetworkPolicy and its regeneration after manual deletion are verified, demonstrating the *drift protection* mechanism.

Additionally, an integral test is executed that deploys a pod with minimal configuration and verifies that the set of active mutation policies transforms it into a pod with a complete security profile. This test constitutes the most representative demonstration of the central concept of the work: the *auto-hardening* that is automatic and transparent to the developer.

4.7 Comparison with OPA Gatekeeper

The project includes a practical comparison between Kyverno and OPA Gatekeeper. To this end, a representative subset of the policies already defined in Kyverno will be reimplemented in OPA, with the objective of illustrating differences in syntax, capabilities and ease of use. The comparison does not seek to determine which tool is superior in absolute terms, but rather to evaluate which adapts better to the specific needs of an automated DevSecOps environment where mutation and generation of resources are key requirements.

This practical exercise complements the theoretical comparison of Chapter 2 and allows drawing conclusions grounded in direct experience with both tools.

4.8 Scope decisions and anticipated limitations

Throughout the planning process, several explicit decisions have been made about the project scope that are worth documenting:

Sysdig Secure. The supervisor warned that Sysdig Secure is a SaaS service that has historically presented integration issues with virtual machines, sometimes requiring cloud infrastructure to work correctly. Its integration is considered if viable with the laboratory environment,

but it is accepted in advance that it may be necessary to document the limitation and replace it with an analysis of the runtime security capabilities the tool would offer in a real cloud environment.

Ansible. Although there is interest in incorporating Ansible to automate cluster provisioning and component installation, the supervisor recommended addressing it only if the rest of the work is complete. Ansible is considered an optional extension that does not condition the main deliverable of the project.

Advanced image validation. Digital signature verification functionalities (Cosign, Notary) and metadata inspection via `imageRegistry` remain outside the committed scope, although they will be included if time permits. Their absence does not affect the demonstration of the Policy-as-Code model, which is sufficiently covered by the main set of policies.

These decisions reflect realistic planning aimed at guaranteeing a solid deliverable within the available time, rather than covering an excessive scope with the risk of remaining incomplete.

5 Architecture Design

This chapter describes the architectural design of the DevSecOps environment implemented in this work. It presents the overall system vision, the role of each component, the complete CI/CD flow with integrated security, the network topology of the laboratory cluster and, finally, the design decisions that have guided the construction of the environment and the technical reasons that justify them.

5.1 General system architecture

The system architecture is organised into three clearly differentiated functional planes: the development plane, the automation plane and the execution plane. This separation is not merely conceptual, but reflects a real division of responsibilities between system components and the moments in the lifecycle when each one acts.

The **development plane** encompasses the set of activities performed by the engineering team: writing application code, defining Kubernetes manifests and formulating security policies. All this production materialises as commits on Git repositories, which constitute the system's source of truth.

The **automation plane** comprises the mechanisms that process changes introduced in the repositories and transform the code into deployable artefacts, verifying their security in the process. This plane encompasses the continuous integration pipeline managed by GitHub Actions, which builds container images and subjects them to vulnerability analysis using Trivy, and the GitOps controller implemented by ArgoCD, which detects differences between the desired state defined in Git and the real state of the cluster and applies the necessary reconciliations.

The **execution plane** corresponds to the Kubernetes cluster, where workloads are finally deployed and where Kyverno acts as the last line of defence, intercepting every resource creation or modification request to apply the defined security policies before any object is persisted in the system.

Figure 5.1 illustrates the global system architecture, showing the relationship between the three planes and the information flows between them.

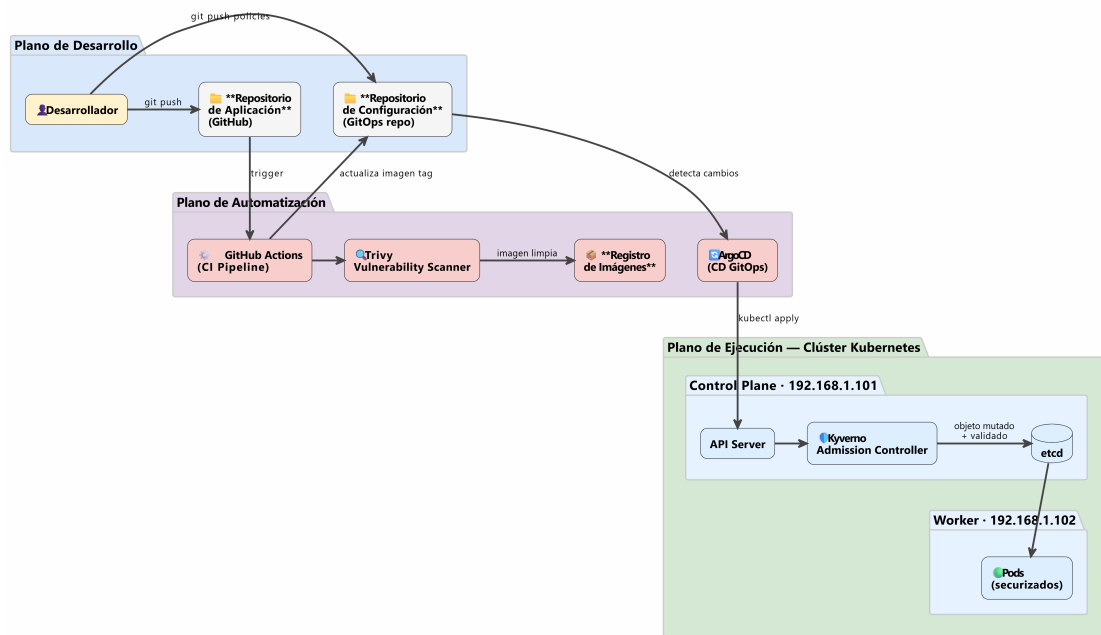


Figure 5.1: General architecture of the DevSecOps environment

A central architectural aspect is the adoption of the **two-repository pattern** for the implementation of the GitOps model. The application repository contains the source code of the workloads and business logic, while the configuration repository — commonly known as the *GitOps repo* — stores Kubernetes manifests, Kyverno policies and infrastructure configurations. This separation allows the CI pipeline to modify only the configuration repository (updating the image tag after a successful build) without ArgoCD needing access to the application source code, reducing the attack surface and clarifying the responsibilities of each component.

5.2 Component description

5.2.1 Kubernetes cluster

The Kubernetes cluster constitutes the execution platform on which the entire system is based. Following the architecture described in Chapter 4, it consists of two nodes: a control node (192.168.1.101) and a worker node (192.168.1.102), both running Ubuntu Server 24.04 LTS on VirtualBox virtual machines.

The **API Server** is the central component of the control plane and the only entry point for all cluster operations. All system tools — ArgoCD, Kyverno, `kubectl` — interact exclusively through the API Server, which exposes a RESTful API and applies authentication, authorisation and admission control mechanisms to each received request [8]. It is precisely

in this component where Kyverno's webhooks are registered to intercept admission traffic.

The **etcd** store persists the complete cluster state as distributed key-value pairs. No resource reaches etcd without having gone through the complete admission process, which guarantees that the persisted state always complies with the defined security policies.

5.2.2 Kyverno

Kyverno is deployed in the dedicated namespace **kyverno** and operates through four independent controllers, each with well-defined responsibilities [10]:

The **Admission Controller** is the critical component for this work. It registers in the API Server via two objects of type **MutatingWebhookConfiguration** and **ValidatingWebhookConfiguration**, which instruct the API Server to route admission requests to Kyverno before persisting them. This controller evaluates *mutate* and *validate* policies in real time, with a latency that must remain below the timeout threshold configured in the webhook (by default, 10 seconds).

The **Background Controller** evaluates policies over resources already existing in the cluster. This is especially relevant for *generate* policies: when the **generate-networkpolicy-metadata** policy is applied, the Background Controller is responsible for retroactively generating **NetworkPolicies** in namespaces that already existed before the policy was deployed.

The **Reports Controller** generates and keeps updated **PolicyReport** and **ClusterPolicyReport** resources, which consolidate the results of evaluating all policies over all cluster resources. These reports are fundamental for compliance auditing.

The **Cleanup Controller** manages the scheduled deletion of resources generated by *generate* policies when the resource that originated them is deleted, maintaining cluster consistency.

5.2.3 ArgoCD

ArgoCD implements the GitOps paradigm by acting as a continuous reconciliation operator between the state declared in the configuration repository and the real state of the cluster [18]. Its internal architecture consists of several sub-components: the *Application Controller* is the core that detects state differences (*diff*) and applies reconciliations; the *Repo Server* manages connections to Git repositories and the rendering of manifests; and the *API Server* exposes the user interface and management API.

In the context of this work, ArgoCD manages Kyverno policies like any other Kubernetes resource: the **ClusterPolicy** objects stored in the configuration repository are automatically

synchronised to the cluster. This means that any policy modification — correction, hardening or new policy — follows the same versioned approval workflow as any infrastructure change, guaranteeing complete traceability.

ArgoCD's *self-healing* capability directly complements Kyverno's *drift protection* mechanism: if someone manually modifies a policy in the cluster, ArgoCD detects the deviation and reverts it to the state defined in Git; if someone deletes a NetworkPolicy generated by Kyverno, the Background Controller of Kyverno regenerates it. Both layers work in a coordinated manner to guarantee that the system state always remains aligned with what is defined in the repository.

5.2.4 GitHub Actions and Trivy

GitHub Actions acts as the continuous integration engine, running automatically on each *push* to the application repository [13]. The pipeline builds the container image, invokes Trivy to perform a vulnerability analysis [14] and, only if the analysis does not detect critical vulnerabilities, publishes the image to the registry and updates the image tag in the configuration repository.

The security of the pipeline itself deserves specific attention. The supervisor's alert about the TeamPCP group attack, which compromised CI/CD pipelines by manipulating third-party GitHub Actions [17], motivates two concrete design decisions detailed in Section 5.5: the anchoring of action versions via SHA hash and the limitation of runner token permissions to the minimum necessary.

5.2.5 Sysdig Secure

Sysdig Secure adds the runtime security dimension to the environment, complementing Kyverno's preventive controls with anomalous behaviour detection during pod execution [15]. As a SaaS service, its integration with the virtual machine-based laboratory environment may present technical limitations, as documented in Chapter 4. Nevertheless, its architectural role is relevant: while Kyverno acts at deployment time, Sysdig Secure acts during execution, covering the attack surface that cannot be statically controlled.

5.3 DevSecOps deployment flow

The complete system flow can be described in two differentiated phases: the continuous integration phase, the responsibility of GitHub Actions, and the continuous deployment phase, the responsibility of ArgoCD and Kyverno. The explicit separation between CI and CD is an intentional architectural decision justified in Section 5.5.

5.3.1 Continuous integration phase

The CI flow begins when a developer performs a *push* to the main branch of the application repository. GitHub Actions detects the event and launches the defined workflow, which executes the following steps sequentially:

First, the container image is built from the repository's Dockerfile. Next, Trivy analyses the resulting image for known vulnerabilities, consulting its CVE database. If the analysis detects critical or high severity vulnerabilities, the pipeline fails at this point and the image is not published, preventing vulnerable code from reaching the execution environment. If the analysis is satisfactory, the image is published to the container registry with a tag that includes the commit hash, guaranteeing traceability between the deployed artefact and the source code that generated it.

The last step of the CI pipeline updates the deployment manifest in the configuration repository, replacing the previous image tag with the new one. This commit in the configuration repository acts as a trigger signal for the CD phase.

Figure 5.2 illustrates this complete CI/CD flow.

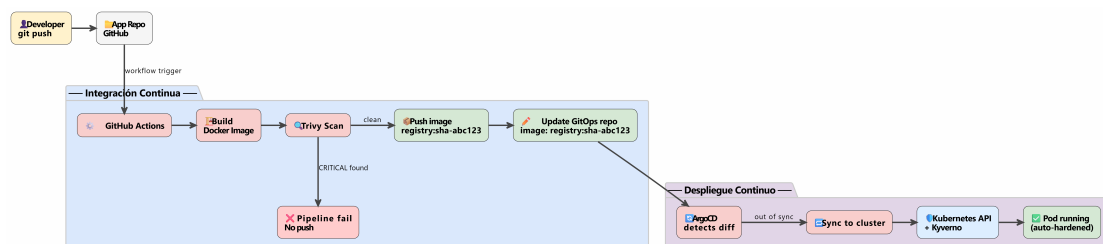


Figure 5.2: Continuous integration and deployment flow with security analysis

5.3.2 Continuous deployment and admission phase

ArgoCD periodically monitors the configuration repository. Upon detecting the new commit introduced by the CI pipeline, it computes the difference between the updated manifest and the current cluster state and initiates synchronisation, sending the resource update request to the Kubernetes API Server.

At this point, Kyverno intervenes through the Admission Controllers mechanism. The Kubernetes admission process follows a determined sequence [8] illustrated in Figure 5.3:

After request authentication and authorisation, the API Server invokes the registered *mutating admission webhooks*. Kyverno receives the object in its original state and sequentially applies all active *mutate* policies: it injects CPU and memory limits and requests (P4), completes

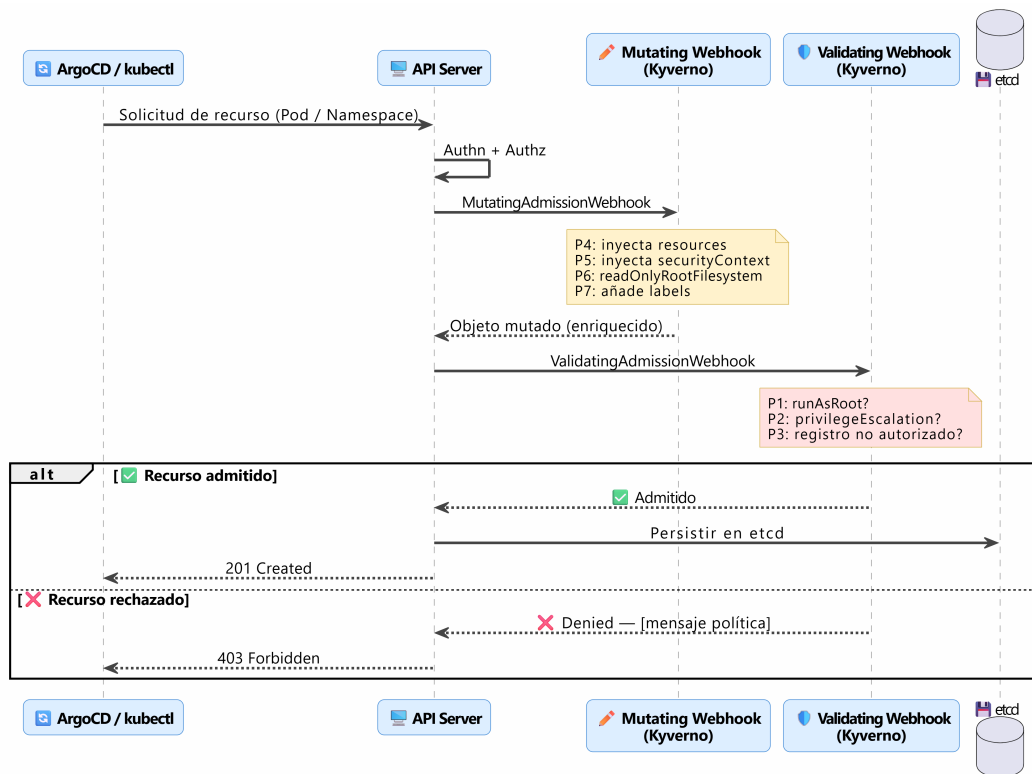


Figure 5.3: Kyverno admission flow: mutation before validation

the container's `securityContext` with the missing secure configurations (P5), forces the read-only filesystem (P6) and adds traceability labels (P7). The object returned to the API Server is already the version enriched with all security configurations.

Next, the API Server invokes the *validating admission webhooks*. Kyverno evaluates the *validate* policies over the already mutated object. Since mutation has previously applied the secure configurations, the vast majority of pods pass this validation without rejection. Validation policies act as a final barrier for cases that mutation cannot resolve: the use of images from unauthorised registries (P3), explicit execution as root user when the developer has deliberately specified it (P1) or explicit privilege escalation request (P2). If any validation policy rejects the object, the API Server returns an error with the message defined in the policy and the resource is not persisted in etcd.

If the object passes all admission webhooks, it is persisted in etcd, the Scheduler determines on which node it should run and the kubelet on the worker node proceeds with pod creation.

5.4 Cluster network architecture

The network topology of the laboratory environment is structured in three layers, as shown in Figure 5.4.

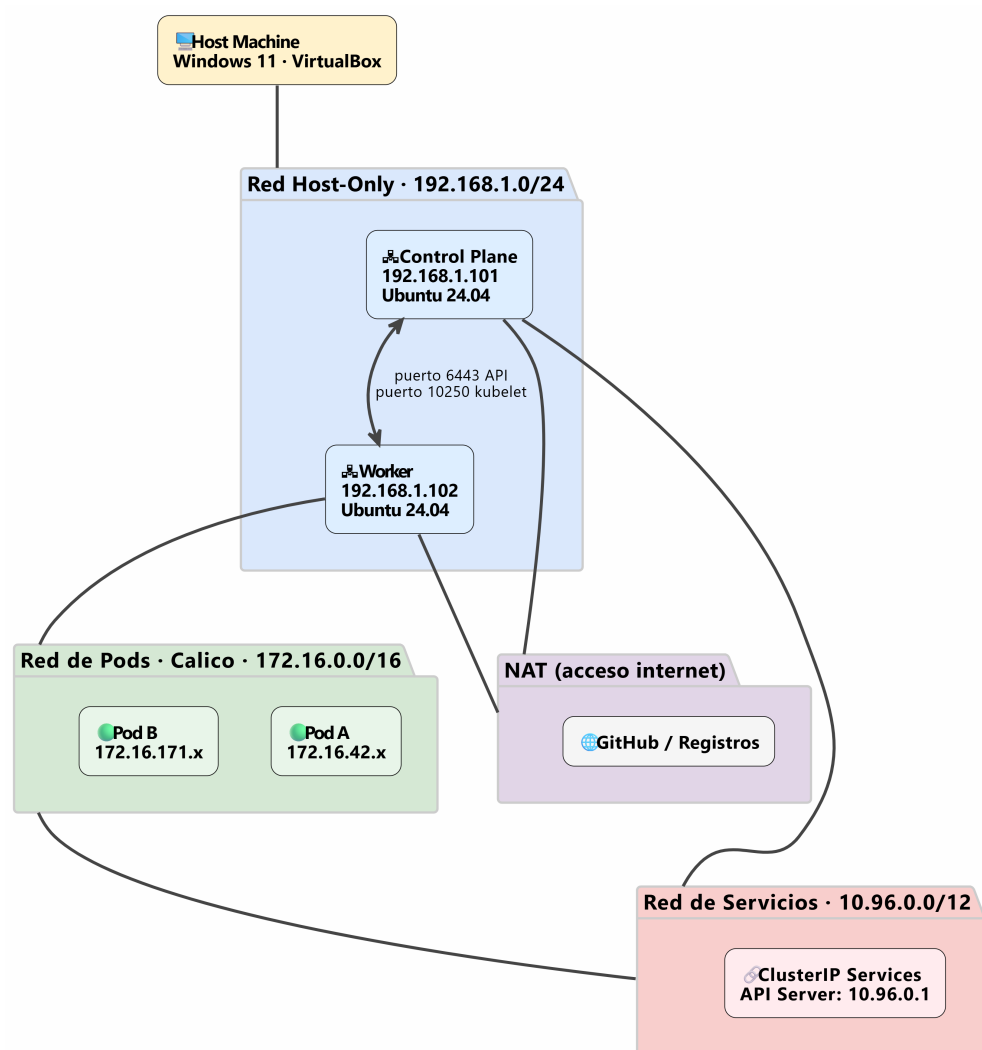


Figure 5.4: Network architecture of the laboratory cluster

The **management layer** corresponds to VirtualBox's *host-only* network (192.168.1.0/24), through which the host machine and both cluster nodes communicate. The nodes have statically assigned IP addresses: 192.168.1.101 for the control node and 192.168.1.102 for the worker node. All internal cluster communication — between the API Server and kubelets, between the API Server and Kyverno and ArgoCD controllers — transits through this network. Additionally, each node has a VirtualBox NAT interface that provides internet access, used for downloading container images and communication with external services such as GitHub or image registries.

The **pod layer** is managed by Calico [16], which assigns a private addressing space (172.16.0.0/16) to cluster pods. Calico implements Kubernetes Network Policies through iptables or eBPF rules, depending on the configuration, guaranteeing that the traffic restrictions defined in the policies are applied effectively. This implementation is a requirement for the project's policy P8: the NetworkPolicy generated by Kyverno to block access to the cloud metadata service (169.254.169.254/32) only takes effect if the underlying CNI plugin respects and applies Kubernetes Network Policies, a condition that Calico natively meets.

The **service layer** uses the addressing space 10.96.0.0/12, reserved by kubeadm for ClusterIP type Service objects. The services of system components — API Server (10.96.0.1), ArgoCD Server, Kyverno — are accessible within the cluster through this address space, while external access to the API Server is achieved directly through the control node IP via port 6443.

5.5 Design decisions

The architectural decisions adopted in the system design are not arbitrary, but respond to specific technical and security requirements. This section documents the most relevant ones and justifies each of them.

5.5.1 Separation between CI and CD

The separation between continuous integration (GitHub Actions) and continuous deployment (ArgoCD) is a deliberate architectural decision, rather than a single pipeline that builds and deploys in one sequence. This separation offers several advantages. First, it decouples the necessary credentials: the CI pipeline needs access to the image registry and configuration repository, but not to the Kubernetes cluster; ArgoCD has access to the cluster but not to the source code. This segregation reduces the blast radius of a possible compromise of either component. Second, it allows applying security controls at the boundary between the two phases: the image is only published if it passes the Trivy analysis, and ArgoCD only deploys

what is in the configuration repository, which acts as an audited control point.

5.5.2 ClusterPolicy scope vs. Policy

Kyverno allows defining policies with cluster scope (`ClusterPolicy`) or namespace scope (`Policy`). In this work, exclusively `ClusterPolicy` objects are used, which guarantees that security policies apply to all cluster namespaces without exceptions. A namespaced policy could inadvertently be omitted when creating a new namespace, creating a coverage gap. The use of `ClusterPolicy` implements the *secure by default* principle: security is applied everywhere unless explicitly excluded through documented exception rules.

5.5.3 Enforce mode vs. Audit mode

Kyverno can operate in two modes: `Enforce`, which actively blocks resources that violate policies, and `Audit`, which allows deployment but records violations in `PolicyReports`. In this work, validation policies operate in `Enforce` mode, which guarantees that no non-compliant resource reaches the cluster. Although in a real production environment it is usually recommended to start in `Audit` mode to avoid disruptions, in the context of this laboratory environment `Enforce` mode is what effectively demonstrates the blocking mechanism and its real effect on deployments.

5.5.4 Mutation before validation as a hardening strategy

The Kubernetes webhook execution order — mutation before validation — is deliberately leveraged as an automatic *hardening* strategy. Mutation policies are not optional complements, but the main security mechanism of the system: instead of rejecting pods with incomplete configurations (which would create friction for developers), Kyverno automatically completes them with the correct security profile. Validation policies act only on configurations that cannot be resolved through mutation, such as the use of unauthorised image registries or the presence of explicitly declared privileges.

This strategy aligns security with usability: the developer can work with minimal manifests and obtain a correctly secured pod without needing to know in detail all Kubernetes security fields. The responsibility for compliance rests on the platform, not on the developer.

5.5.5 synchronize: true in generation policies

Policy P8 uses the `synchronize: true` field in its *generate* type definition. This configuration instructs Kyverno's Background Controller to monitor generated resources and recreate

them automatically if they are deleted or modified in an unauthorised manner. This decision implements the *drift protection* principle at the level of security resources: the NetworkPolicy that blocks access to the IMDS cannot be permanently deleted by an operator (intentionally or accidentally), as Kyverno will regenerate it within seconds. Without this field, generation would be a one-time event and protection could be circumvented.

5.5.6 Pipeline supply chain security

The compromise of the TeamPCP group over CI/CD pipelines [17] illustrates that the automation pipeline itself can become an attack vector. In response to this threat, the pipeline design adopts two concrete measures. The first is the anchoring of GitHub Actions versions via their full SHA hash, rather than floating tags such as `@latest` or `@v3`, which can be redirected to malicious versions without modifying the reference name. The second is the configuration of the GitHub token with minimum permissions (*least privilege*): the runner only has the permissions strictly necessary for its function, limiting the impact of a possible compromise of the execution environment.

5.5.7 Choice of Calico as CNI plugin

The choice of Calico [16] over alternatives such as Flannel or Weave responds to a direct functional requirement: policy P8 requires that Kubernetes Network Policies be effectively applied by the CNI plugin. Flannel, for example, does not natively support Network Policies. Calico, on the other hand, is one of the reference CNI plugins in production environments and provides a complete implementation of the Kubernetes Network Policy model, guaranteeing that the traffic blocking rules generated by Kyverno have a real effect on pod network communication.

6 Cluster Implementation and Security Policies

This chapter documents the practical implementation of the Kubernetes cluster and the policy engine that constitute the security core of the environment designed in Chapter 5. Unlike that chapter, which establishes architectural decisions and their theoretical justification, this chapter focuses on concrete execution: the steps carried out, the deployed policies with their complete manifests, the executed tests and the observed results. The implementation of the DevSecOps pipeline automation components — ArgoCD for continuous deployment and GitHub Actions for continuous integration — is addressed in the following chapters.

The chapter is structured into seven sections. The first covers Kubernetes cluster provisioning on the laboratory environment. The second describes Kyverno deployment and the verification of its integration with the API Server. The third, fourth and fifth sections document respectively the implementation of validation, mutation and generation policies, with their manifests, technical justifications and associated tests. The sixth section collects the most relevant operational considerations and empirical findings of the process, including the verification of webhook execution order and the integral *auto-hardening* test. The chapter closes with the seventh section, dedicated to Policy Reports as the native compliance audit mechanism.

6.1 Kubernetes cluster provisioning

The starting point of the project is the provisioning of a functional Kubernetes cluster on the laboratory environment described in Section 4.2. The virtual machines, provided by the supervisor, run Ubuntu Server 24.04 LTS and are pre-configured with the necessary packages (`kubeadm`, `kubelet`, `kubectl`) and with the VirtualBox network layer in NAT plus *host-only adapter* mode.

The control plane is initialised from the `control` node using the following command:

```
sudo kubeadm init --apiserver-advertise-address=192.168.1.101
```

The `--apiserver-advertise-address` parameter is necessary in environments with multiple network interfaces to indicate to `kubeadm` on which address the API Server should listen. Without this parameter, `kubeadm` would select the default interface, which in VirtualBox corresponds to the NAT adapter (`10.0.2.x`), inaccessible from the worker node. By specifying the *host-only* interface address, it is guaranteed that cluster components — including Kyverno’s webhooks — are reachable through the laboratory’s internal network.

After initialisation, `kubectl` access is configured for the `student` user by copying the configuration file generated by `kubeadm`:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Next, the worker node (192.168.1.102) joins the cluster via the `kubeadm join` command generated during initialisation, which includes the bootstrap token and the API Server certificate hash to guarantee connection authenticity.

Once both nodes form part of the cluster, the Calico CNI plugin version 3.28 is installed:

```
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/
↪ v3.28.0/manifests/calico.yaml
```

Calico configures the pod addressing space with the CIDR 172.16.0.0/16, the default value for this distribution. Once installation is complete and after system component convergence, the cluster reaches operational state with both nodes in Ready state.

6.2 Kyverno deployment and integration

With the cluster operational, the next step is the installation of Kyverno as the security policy engine. Kyverno is deployed via Helm in the dedicated namespace `kyverno`:

```
helm repo add kyverno https://kyverno.github.io/kyverno/
helm repo update
helm install kyverno kyverno/kyverno -n kyverno --create-namespace
```

After installation, the verification of pod state in the `kyverno` namespace confirms that the four controllers are active and in `Running` state, as shown in Figure 6.1.

```
student@control:~$ kubectl get pods -n kyverno
NAME                                READY   STATUS    RESTARTS   AGE
kyverno-admission-controller-659d58644b-hvb5x   1/1     Running   0          11m
kyverno-background-controller-778bffc669-x9fsg   1/1     Running   0          11m
kyverno-cleanup-controller-8bfc4f578-ll5gd      1/1     Running   0          11m
kyverno-reports-controller-6c666d96-wjwrv       1/1     Running   0          11m
student@control:~$
```

Figure 6.1: Kyverno controllers deployed in the cluster

The four controllers — `admission-controller`, `background-controller`, `cleanup-controller` and `reports-controller` — operate independently and complementarily, as described in Section 5.2.2. The most relevant controller for the system's real-time behaviour is

the *admission controller*, which registers in the API Server immediately after its deployment.

The verification of registered webhooks confirms that Kyverno has correctly established its integration with the API Server. The query of `ValidatingWebhookConfiguration` objects shows seven configurations registered by Kyverno, as seen in Figure 6.2.

```
student@control:~$ kubectl get validatingwebhookconfigurations
NAME                                                                 WEBHOOKS   AGE
kyverno-cel-exception-validating-webhook-cfg                       1          28d
kyverno-cleanup-validating-webhook-cfg                             1          28d
kyverno-exception-validating-webhook-cfg                           1          28d
kyverno-global-context-validating-webhook-cfg                      1          28d
kyverno-policy-validating-webhook-cfg                              1          28d
kyverno-resource-validating-webhook-cfg                            1          28d
kyverno-ttl-validating-webhook-cfg                                  1          28d
student@control:~$
```

Figure 6.2: `ValidatingWebhookConfigurations` registered by Kyverno in the API Server

Among the registered configurations, the most relevant for this work is `kyverno-resource-validating-webhook-cfg`, which intercepts creation and modification requests for arbitrary cluster resources and routes them to Kyverno’s *admission controller* for evaluation. The additional configurations cover specific cases such as policy validation itself (`kyverno-policy-validating-webhook-cfg`), global context (`kyverno-global-context-validating-webhook-cfg`) and exception and TTL management. Similarly, Kyverno registers the corresponding `MutatingWebhookConfiguration` objects for processing *mutate* policies.

With Kyverno installed and its webhooks active, the cluster is ready to receive security policies. Figure 6.3 shows the cluster state at this point, before applying any policy: the `ClusterPolicy` query returns no resources.

```
student@control:~$ kubectl get clusterpolicies
No resources found
student@control:~$
```

Figure 6.3: Initial cluster state: no policies deployed

6.2.1 Kyverno’s autogen mechanism

Before documenting the implemented policies, it is necessary to understand a mechanism that conditions the real scope of all of them: automatic rule generation, known as *autogen*.

The problem: Pods are rarely created directly

The eight policies in this work declare `match.resources.kinds: [Pod]`, which appears to restrict their application to Pod type resources. However, in practice Pods are not created directly: they are generated by higher-level controllers — `Deployment`, `StatefulSet`, `DaemonSet`, `Job`, `CronJob`, `ReplicaSet` — through a `spec.template` field that contains the specification of the Pod to be instantiated.

If Kyverno only evaluated resources of type Pod, a developer — or an attacker — could create a `Deployment` with insecure configuration without the policy intercepting it. The Pods derived from that `Deployment` would be individually rejected by the *admission webhook* when the controller tried to create them, but the `Deployment` object would remain persisted in etcd and the controller would enter a loop of recurring errors, generating operational noise and leaving the root resource in an inconsistent state. In permissive failure modes, depending on the webhook's `failurePolicy` configuration, some resources might even manage to execute.

The solution: auto-generated rules

Kyverno solves this problem transparently for the policy author. When processing a `ClusterPolicy` that declares `match.kinds: [Pod]`, the engine analyses the defined pattern and automatically generates additional rules — called *autogen rules* — that apply the same control over the `spec.template.spec` of controllers that create Pods. The developer defines the policy once and Kyverno internally replicates it to cover `Deployment`, `StatefulSet`, `DaemonSet`, `Job`, `CronJob` and `ReplicaSet`.

Generation distinguishes two cases depending on nesting depth. For most controllers, the auto-generated rule is called `autogen-<original-name>` and applies the pattern over `spec.template.spec`. For `CronJob`, whose Pod specification is one level deeper (`spec.jobTemplate.template.spec`), Kyverno generates a second additional rule, `autogen-cronjob-<original-name>`, which adjusts the pattern path accordingly.

The empirical verification of this behaviour is carried out by inspecting the `ClusterPolicy` object once deployed:

```
kubectl get clusterpolicy disallow-root -o yaml | grep -A 50 "autogen:"
```

The output of this command shows the rules `autogen-require-run-as-non-root`, covering `Deployments`, `StatefulSets`, `Jobs` and `CronJobs`, and `autogen-cronjob-require-run-as-non-root`, handling the specific `CronJob` case. These rules are generated internally by Kyverno and

stored in the policy object in etcd; they do not require any manual intervention from the operator.

Differentiated behaviour: **validate** vs. **mutate**

The autogen mechanism does not behave identically for all policy types:

- For **validate** policies, autogen is always automatic and unconditional. Kyverno generates additional rules for all controllers that create Pods without the need for any explicit configuration.
- For **mutate** policies using `patchStrategicMerge` — such as the four mutation policies in this work (P4, P5, P6, P7) — autogen is also automatic. Kyverno adapts the patch so that it applies over the controller's `spec.template.spec` field, guaranteeing that mutations injected on direct Pods are equally replicated on Pods derived from Deployments or StatefulSets.
- For **mutate** policies using `patchesJson6902`, autogen is not applied by default, since the absolute JSON paths of this mechanism are not automatically generalisable. In these cases, the operator must explicitly configure the autogen behaviour.

Implication for this work's policies

The eight implemented policies — P1 to P8 — declare `match.kinds: [Pod]`, but thanks to the autogen mechanism they effectively cover Deployments, StatefulSets, Jobs and CronJobs. This means that the real scope of the security system is significantly greater than what can be derived from a direct reading of the manifests.

This behaviour became evident during the initial implementation phases: PolicyViolations generated by the Reports Controller included messages such as `autogen-require-run-as-non-root` being applied to `Deployment` and `ReplicaSet` objects of Kyverno's own components and `kube-system`, confirming that coverage automatically extended to higher-level resources without having declared those kinds in the policy.

Kyverno allows controlling this behaviour through the annotation `pod-policies.kyverno.io/autogen-controllers` in the policy's metadata, which accepts an explicit list of controllers to cover or the value `none` to completely disable autogen. In this work the default behaviour is used, which offers the broadest possible coverage without additional configuration.

6.3 Validation policies

Validation policies constitute the reactive layer of the security system: they act by rejecting resources that violate conditions that cannot be resolved through automatic mutation. As justified in Section 5.5.4, mutation policies have precedence and act first; validation policies therefore operate as a final barrier for explicitly insecure configurations that should not be silently corrected, but rejected with a clear message to the requester.

6.3.1 Policy P1: disallow-root

The first validation policy prevents the deployment of pods that do not explicitly specify that their containers must run as a non-root user:

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-root
spec:
  validationFailureAction: Enforce
  background: true
  rules:
    - name: require-run-as-non-root
      match:
        any:
          - resources:
              kinds:
                - Pod
      validate:
        message: "Containers must not run as root"
        pattern:
          spec:
            securityContext:
              runAsNonRoot: true
```

The policy operates through *pattern matching*: Kyverno verifies that the field `spec.securityContext.runAsNonRoot` exists with the value `true`. If the field is absent or is `false`, the webhook returns a validation error with the defined message and the request is rejected.

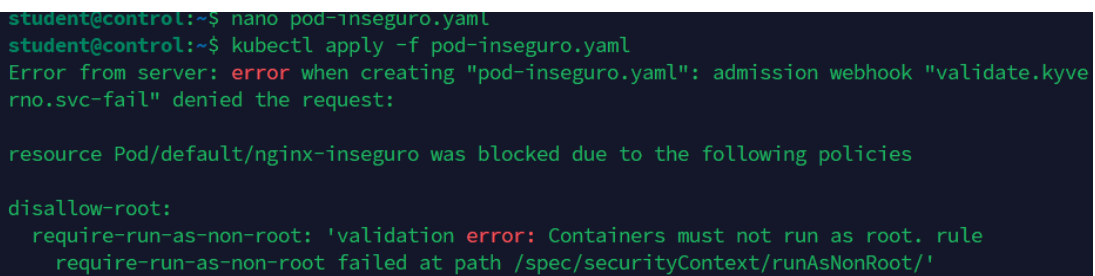
The technical justification for this control is twofold. On one hand, a process with UID `0` inside the container inherits the complete set of Linux capabilities assigned to it, including

CAP_SYS_ADMIN and CAP_NET_RAW, which significantly expand the attack surface. On the other hand, if a container isolation escape occurs — through a runtime or kernel vulnerability — a process that was already running as root in the container will obtain root privileges on the host node, potentially compromising the entire cluster.

The validation test applies the following pod, which specifies no `securityContext`:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-insecure
spec:
  containers:
  - name: nginx
    image: nginx
```

The result, shown in Figure 6.4, confirms that Kyverno’s webhook rejects the request indicating the exact path of the failing field.



```
student@control:~$ nano pod-inseguro.yaml
student@control:~$ kubectl apply -f pod-inseguro.yaml
Error from server: error when creating "pod-inseguro.yaml": admission webhook "validate.kyverno.svc-fail" denied the request:

resource Pod/default/nginx-inseguro was blocked due to the following policies

disallow-root:
  require-run-as-non-root: 'validation error: Containers must not run as root. rule
    require-run-as-non-root failed at path /spec/securityContext/runAsNonRoot/'
```

Figure 6.4: P1: pod without `runAsNonRoot` blocked by Kyverno’s webhook

6.3.2 Policy P2: disallow-privilege-escalation

The second policy requires all containers to explicitly declare `allowPrivilegeEscalation: false` in their container-level `securityContext`:

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-privilege-escalation
spec:
  validationFailureAction: Enforce
  rules:
  - name: no-privilege-escalation
    match:
```

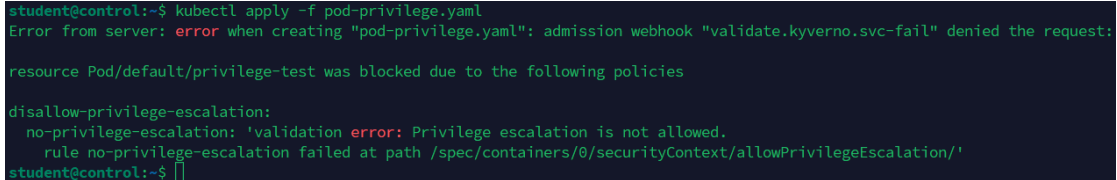
```

any:
  - resources:
      kinds:
        - Pod
  validate:
    message: "Privilege escalation is not allowed"
    pattern:
      spec:
        containers:
          - securityContext:
              allowPrivilegeEscalation: false

```

This field controls the `no_new_privs` flag of the Linux kernel [19], which prevents any child process from acquiring more privileges than its parent process. Without this flag active, an unprivileged UID process could invoke a binary with the SUID bit active — such as `sudo` — to elevate its privileges to root, circumventing `runAsNonRoot` controls.

The test applies a pod that explicitly requests privilege escalation. The result, in Figure 6.5, shows rejection with the path of the offending field.



```

student@control:~$ kubectl apply -f pod-privilege.yaml
Error from server: error when creating "pod-privilege.yaml": admission webhook "validate.kyverno.svc-fail" denied the request:
resource Pod/default/privilege-test was blocked due to the following policies
disallow-privilege-escalation:
  no-privilege-escalation: 'validation error: Privilege escalation is not allowed.
    rule no-privilege-escalation failed at path /spec/containers/0/securityContext/allowPrivilegeEscalation/'
student@control:~$

```

Figure 6.5: P2: pod with `allowPrivilegeEscalation: true` blocked

It is worth noting the interaction between this policy and mutation policy P5. Since *mutating webhooks* are invoked before *validating webhooks*, P5 automatically injects `allowPrivilegeEscalation: false` in pods that do not specify this field. Consequently, P2 acts only as a final barrier for cases where the value `true` has been explicitly declared — an insecure intention declaration that should not be silently corrected, but rejected.

6.3.3 Policy P3: restrict-image-registries

The third policy restricts image registries to a set of verified sources, implemented via Kyverno's `foreach` mechanism:

```

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: restrict-image-registries

```

```
spec:
  validationFailureAction: Enforce
  background: true
  rules:
  - name: validate-registries
    match:
      any:
        - resources:
            kinds:
              - Pod
    validate:
      message: >-
        Only images from authorised registries are allowed:
        docker.io, ghcr.io, registry.k8s.io, quay.io.
        The image {{ element.image }} is not permitted.
      foreach:
        - list: "request.object.spec.containers"
          deny:
            conditions:
              all:
                - key: "{{ element.image }}"
                  operator: AnyNotIn
                  value:
                    - "docker.io/*"
                    - "ghcr.io/*"
                    - "registry.k8s.io/*"
                    - "quay.io/*"
                    - "nginx*"
                    - "alpine*"
                    - "busybox*"
                    - "nginxinc/*"
```

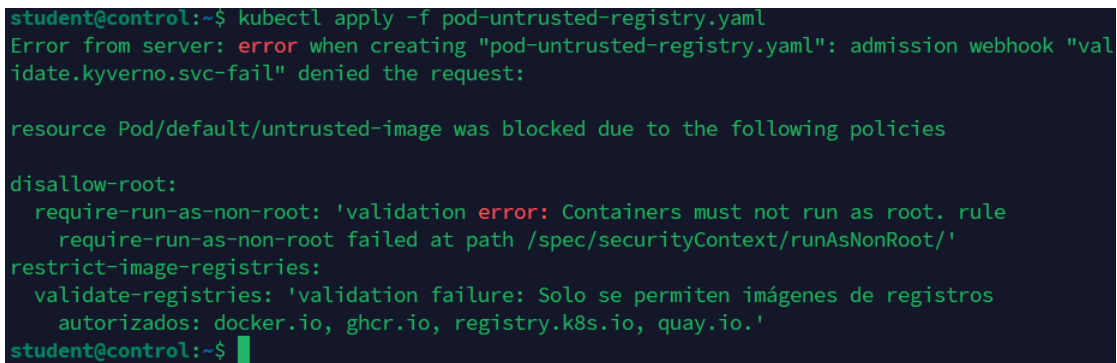
The policy iterates over all pod containers and denies the request if any image does not belong to the authorised patterns. Patterns without registry prefix (`nginx*`, `alpine*`, `busybox*`) are necessary because Kubernetes implicitly resolves unprefixed references to Docker Hub. The inclusion of `ghcr.io/*` is an operational requirement, given that Kyverno and other cluster component images are distributed through that registry.

This policy constitutes a first line of defence against supply chain attacks of the type documented by the TeamPCP group [17]: an attacker who hosts a malicious image on an arbitrary registry

will not be able to run it in the cluster without modifying this policy. In mature production environments, the control would be complemented with digital signature verification using Cosign or Notary, considered future work in Section 4.7.

During implementation, a relevant engine behaviour was observed: the variable `{{ element.image }}` referenced in the top-level message field does not expand at rejection time, since that variable only has scope within the `foreach` block. Kyverno accepts the policy as valid and the `READY` field shows `True`, but at rejection time the error message only shows the static part, omitting the specific image name. This behaviour illustrates Kyverno's variable scoping model and must be taken into account when designing informative error messages in policies with `foreach`.

The validation test applies a pod with an image from an unauthorised registry. The result, in Figure 6.6, reveals a defence-in-depth behaviour: the pod is simultaneously blocked by two independent policies.



```
student@control:~$ kubectl apply -f pod-untrusted-registry.yaml
Error from server: error when creating "pod-untrusted-registry.yaml": admission webhook "validation.kyverno.svc-fail" denied the request:

resource Pod/default/untrusted-image was blocked due to the following policies

disallow-root:
  require-run-as-non-root: 'validation error: Containers must not run as root. rule
    require-run-as-non-root failed at path /spec/securityContext/runAsNonRoot/'
restrict-image-registries:
  validate-registries: 'validation failure: Solo se permiten imágenes de registros
    autorizados: docker.io, ghcr.io, registry.k8s.io, quay.io.'
student@control:~$
```

Figure 6.6: P3: simultaneous blocking by `disallow-root` and `restrict-image-registries`

This result demonstrates that the policy system acts cumulatively: the same resource can violate multiple policies simultaneously, and the error message returned enumerates all detected violations. The image `evilregistry.io/backdoor:latest` is blocked both for not specifying `runAsNonRoot` (P1) and for coming from an unauthorised registry (P3), two independent risk vectors detected in a single response.

6.4 Mutation policies

Mutation policies constitute the main security mechanism of the system, aligning with the central hypothesis of the work: that the automatic correction of insecure configurations at admission time offers greater resilience and better development experience than mere rejection. Instead of blocking pods with incomplete configurations, Kyverno automatically enriches them with the correct security profile, making *hardening* transparent to the developer.

The `patchStrategicMerge` mechanism, used by the four policies in this block, performs an intelligent merge of the original object with the defined patch: it adds absent fields and respects values already explicitly declared by the developer, guaranteeing that mutations are additive and non-destructive.

6.4.1 Policy P4: add-default-resources

The fourth policy automatically injects CPU and memory requests and limits in all containers that do not declare them:

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: add-default-resources
spec:
  rules:
  - name: add-resources
    match:
      any:
      - resources:
          kinds:
            - Pod
    mutate:
      patchStrategicMerge:
        spec:
          containers:
            - (name): "*"
              resources:
                requests:
                  cpu: "100m"
                  memory: "128Mi"
                limits:
                  cpu: "500m"
                  memory: "256Mi"
```

The simultaneous injection of requests and limits with the same values — rather than only limits — responds to a technically justified decision. Kubernetes classifies pods into three Quality of Service classes (*QoS*) [20]: *Guaranteed* (when `requests == limits` for all containers), *Burstable* and *BestEffort*. A pod with *Guaranteed* QoS is the last candidate for eviction when the node experiences resource pressure. Additionally, without requests

defined, the Scheduler lacks information for *bin-packing* decisions and may schedule pods on nodes without sufficient capacity, resulting in OOM failures at runtime.

The test deploys a pod without any `resources` field defined. As shown in Figure 6.7, Kyverno automatically injects `requests` and `limits` values, visible in the `kubectl get pod -o yaml` output.

```
student@control:~$ kubectl apply -f pod-sin-resources.yaml
pod/test-mutate-resources created
student@control:~$ kubectl get pod test-mutate-resources -o yaml | grep -A 8 resources
    {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{},"name":"test-mutate-resources","namespace":"default"},"spec":{"containers":[{"image":"nginxinc/nginx-unprivileged","name":"nginx","securityContext":{"allowPrivilegeEscalation":false,"runAsNonRoot":true,"runAsUser":1000}}],"securityContext":{"runAsNonRoot":true}}}
  creationTimestamp: "2026-04-14T11:07:52Z"
  generation: 1
  labels:
    security.tfm/hardened-by: kyverno
    security.tfm/policy-version: v1.0
  name: test-mutate-resources
  namespace: default
  resourceVersion: "164195"
  uid: 8e140d31-b02c-4538-a77d-af65bde8b45c
spec:
  containers:
  - image: nginxinc/nginx-unprivileged
    imagePullPolicy: Always
    name: nginx
    resources:
      limits:
        cpu: 500m
        memory: 256Mi
      requests:
        cpu: 100m
        memory: 128Mi
    securityContext:
      allowPrivilegeEscalation: false
  --
  resources:
    limits:
      cpu: 500m
      memory: 256Mi
    requests:
      cpu: 100m
      memory: 128Mi
```

Figure 6.7: P4: automatic injection of `resources.requests` and `resources.limits`

It is especially significant that the output also shows the labels `security.tfm/hardened-by: kyverno` and `security.tfm/policy-version: v1.0`, evidence that P7 has also acted on the same resource in the same admission cycle. This confirms that multiple mutation policies are applied in a coordinated manner to each object in a single admission pass.

6.4.2 Policy P5: add-default-securitycontext

The fifth policy injects a secure `securityContext` in all containers that do not specify one at the container level:

```
apiVersion: kyverno.io/v1
```



```
kind: ClusterPolicy
metadata:
  name: add-default-securitycontext
spec:
  rules:
  - name: add-container-security-context
    match:
      any:
      - resources:
          kinds:
          - Pod
    mutate:
      patchStrategicMerge:
        spec:
          containers:
          - (name): "*"
            securityContext:
              allowPrivilegeEscalation: false
              capabilities:
                drop:
                - ALL
```

This policy implements two critical *hardening* controls automatically. The first, `allowPrivilegeEscalation: false`, activates the kernel's `no_new_privs` flag, preventing privilege escalation through SUID binaries. Here the policy acts *proactively* — injecting the secure configuration before reaching validation — in contrast to P2, which acts *reactively* by rejecting the explicit declaration of the insecure value.

The second control, `capabilities.drop: ALL`, removes the complete set of Linux capabilities inherited by default. Without additional configuration, a container inherits capabilities such as `NET_RAW` (raw packet construction), `MKNOD` (block device creation) or `CHOWN` (file ownership change), which unnecessarily expand the attack surface. By dropping them all by default, developers are forced to explicitly declare the capabilities their applications actually need.

This policy is the most representative example of the *auto-hardening* concept: the developer can deploy a pod with minimal configuration without deep knowledge of Kubernetes security fields, and the platform guarantees a solid security profile in a completely transparent manner.

The test deploys a pod with `securityContext` defined only at the pod level. Fig-

ure 6.8 shows the result: the container-level `securityContext` contains `allowPrivilegeEscalation: false`, `capabilities.drop: [ALL]` and `readOnlyRootFilesystem: true` — the latter injected by P6 in the same admission cycle.

```
student@control:~$ nano pod-sin-secontext.yaml
student@control:~$ kubectl apply -f pod-sin-secontext.yaml

# Verificar la inyección del securityContext
kubectl get pod test-mutate-secontext -o yaml | grep -A 6 securityContext
pod/test-mutate-secontext created
  {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{},"name":"test-mutate-secontext","na
mespace":"default"},"spec":{"containers":[{"command":["sleep","3600"],"image":"alpine","name":"alpine"
}],"securityContext":{"runAsNonRoot":true,"runAsUser":1000}}}
  creationTimestamp: "2026-04-14T11:14:55Z"
  generation: 1
  labels:
    security.tfm/hardened-by: kyverno
    security.tfm/policy-version: v1.0
  name: test-mutate-secontext
--
  securityContext:
    allowPrivilegeEscalation: false
    capabilities:
      drop:
        - ALL
    readOnlyRootFilesystem: true
    terminationMessagePath: /dev/termination-log
--
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    serviceAccount: default
    serviceAccountName: default
    terminationGracePeriodSeconds: 30
    tolerations:
student@control:~$
```

Figure 6.8: P5: automatic injection of `allowPrivilegeEscalation` and `capabilities.drop`

6.4.3 Policy P6: add-readonly-rootfs

The sixth policy forces the root filesystem of all containers to read-only mode:

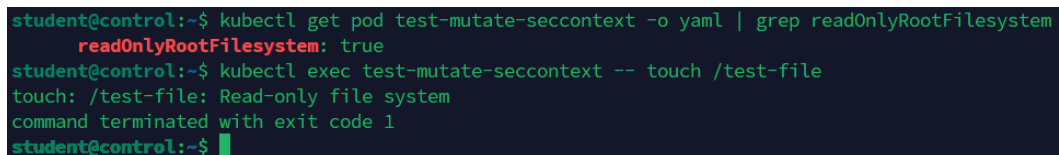
```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: add-readonly-rootfs
spec:
  rules:
    - name: add-readonly-filesystem
      match:
        any:
          - resources:
              kinds:
                - Pod
      mutate:
```

```
patchStrategicMerge:
  spec:
    containers:
      - (name): "*"
        securityContext:
          readOnlyRootFilesystem: true
```

Most post-exploitation techniques after compromising a container — downloading reverse shells, installing cryptominers, executing reconnaissance tools — require writing files to the container filesystem. A read-only root filesystem makes these attempts fail immediately, containing the blast radius of the intrusion.

The test verifies this effect in two steps, shown in Figure 6.9. First, the `kubectl get pod -o yaml` output confirms that the `readOnlyRootFilesystem: true` field has been injected. Second, a command is executed inside the container that attempts to create a file in the root directory:

```
kubectl exec test-mutate-secontext -- touch /test-file
touch: /test-file: Read-only file system
command terminated with exit code 1
```



```
student@control:~$ kubectl get pod test-mutate-secontext -o yaml | grep readOnlyRootFilesystem
readOnlyRootFilesystem: true
student@control:~$ kubectl exec test-mutate-secontext -- touch /test-file
touch: /test-file: Read-only file system
command terminated with exit code 1
student@control:~$
```

Figure 6.9: P6: verification of read-only filesystem at runtime

This result is especially valuable: it demonstrates that the mutation applied by Kyverno at admission time has a real and verifiable effect on container behaviour at runtime, not just on the manifest stored in etcd.

A known operational limitation is that certain applications require writing to directories such as `/tmp` or `/var/cache`. The correct solution — applied in the integral test of Section 6.6.2 — consists of mounting an `emptyDir` volume at the specific paths that require write access. This practice does not weaken the policy, but forces developers to be explicit about their write requirements, which is itself a good container design practice.

6.4.4 Policy P7: add-default-labels

The seventh policy injects traceability labels in all pods processed by the system:

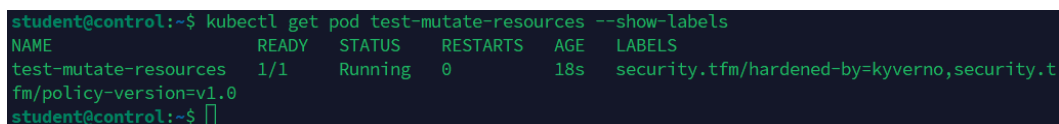
```

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: add-default-labels
spec:
  rules:
  - name: add-security-labels
    match:
      any:
      - resources:
          kinds:
          - Pod
    mutate:
      patchStrategicMerge:
        metadata:
          labels:
            security.tfm/hardened-by: kyverno
            security.tfm/policy-version: "v1.0"

```

Although less obvious than direct security policies, this policy implements an observability capability relevant in production environments. The injected labels allow identifying at any time which pods have been processed by the system and with which policy version, facilitating coverage auditing. The `security.tfm/policy-version` field acts as a marker that allows correlating the applied security profile with the policy version in force at the time of deployment.

Verification via `kubectl get pods --show-labels`, shown in Figure 6.10, confirms that the labels are present in all active pods in the `default` namespace.



```

student@control:~$ kubectl get pod test-mutate-resources --show-labels
NAME                READY   STATUS    RESTARTS   AGE   LABELS
test-mutate-resources 1/1     Running   0           18s   security.tfm/hardened-by=kyverno,security.tfm/policy-version=v1.0
student@control:~$

```

Figure 6.10: P7: traceability labels automatically injected in pods

6.5 Generation policy

6.5.1 Policy P8: generate-networkpolicy-metadata

The eighth policy demonstrates the third type of action available in Kyverno — *generate* — with no functional equivalent in OPA Gatekeeper. Unlike previous policies, which act

on resources in the process of creation, this one reacts to the creation of a Namespace and automatically generates a derived resource: a NetworkPolicy that blocks access to the cloud instance metadata service (IMDS):

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: generate-networkpolicy-metadata
spec:
  rules:
  - name: block-metadata-service
    match:
      any:
      - resources:
          kinds:
          - Namespace
    generate:
      apiVersion: networking.k8s.io/v1
      kind: NetworkPolicy
      name: block-cloud-metadata
      namespace: "{{request.object.metadata.name}}"
      synchronize: true
      data:
        spec:
          podSelector: {}
          policyTypes:
          - Egress
          egress:
          - to:
              - ipBlock:
                  cidr: 0.0.0.0/0
                  except:
                  - 169.254.169.254/32
```

The generated NetworkPolicy allows all outgoing traffic (`cidr: 0.0.0.0/0`) except towards the address 169.254.169.254/32. This IP corresponds to the IMDS present in the main cloud providers (AWS, GCP, Azure), accessible from any instance without prior authentication. An attacker who compromises a pod can make a request to that endpoint to obtain temporary IAM credentials, service tokens and underlying instance network configuration, enabling escalation from the pod to the cloud account. This vector was documentedly used in the

Capital One incident in 2019, where SSRF was exploited to access the IMDS and exfiltrate sensitive data [21].

The `synchronize: true` field activates the Background Controller's *drift protection* mechanism: if the generated NetworkPolicy is modified or deleted, Kyverno automatically restores it.

The first test, shown in Figure 6.11, demonstrates automatic generation: namespace `test-networkpolicy` is created and just 9 seconds later the NetworkPolicy `block-cloud-metadata` exists in that namespace without having been manually applied. The detailed description of the generated resource confirms that Kyverno has added the corresponding traceability labels and that the egress rule exclusively blocks IP `169.254.169.254/32`.

```
student@control:~$ kubectl create namespace test-networkpolicy
namespace/test-networkpolicy created
student@control:~$ kubectl get networkpolicies -n test-networkpolicy
NAME                                POD-SELECTOR  AGE
block-cloud-metadata               <none>        9s
student@control:~$ kubectl describe networkpolicy block-cloud-metadata -n test-networkpolicy
Name:                                block-cloud-metadata
Namespace:                          test-networkpolicy
Created on:                          2026-04-14 11:32:13 +0000 UTC
Labels:                              app.kubernetes.io/managed-by=kyverno
                                     generate.kyverno.io/policy-name=generate-networkpolicy-metadata
                                     generate.kyverno.io/policy-namespace=
                                     generate.kyverno.io/rule-name=block-metadata-service
                                     generate.kyverno.io/trigger-group=
                                     generate.kyverno.io/trigger-kind=Namespace
                                     generate.kyverno.io/trigger-namespace=
                                     generate.kyverno.io/trigger-uid=59c08a41-a46a-469e-9503-e71e24500e73
                                     generate.kyverno.io/trigger-version=v1
Annotations:                        <none>
Spec:
  PodSelector:                      <none> (Allowing the specific traffic to all pods in this namespace)
  Not affecting ingress traffic
  Allowing egress traffic:
    To Port: <any> (traffic allowed to all ports)
    To:
      IPBlock:
        CIDR: 0.0.0.0/0
        Except: 169.254.169.254/32
    Policy Types: Egress
student@control:~$
```

Figure 6.11: P8: NetworkPolicy automatically generated upon namespace creation

The second test, shown in Figure 6.12, demonstrates the *drift protection* mechanism: the NetworkPolicy is manually deleted via `kubectl delete` and, 57 seconds later, it has been automatically regenerated by the Background Controller, without any human intervention.

This behaviour illustrates a fundamental difference compared to OPA Gatekeeper, which lacks native support for resource generation and cannot implement this pattern of continuous protection.

```
student@control:~$ kubectl delete networkpolicy block-cloud-metadata -n test-networkpolicy
networkpolicy.networking.k8s.io "block-cloud-metadata" deleted from test-networkpolicy namespace
student@control:~$ kubectl get networkpolicies -n test-networkpolicy
NAME                                POD-SELECTOR  AGE
block-cloud-metadata               <none>        57s
student@control:~$
```

Figure 6.12: P8: *drift protection* — NetworkPolicy regenerated 57 s after manual deletion

6.6 Operational considerations and lessons learned

This section collects the most relevant operational findings that emerged during implementation and that do not fit in the individual documentation of each policy. Some of them are directly related to the autogen mechanism described in Section 6.2.1: the automatic extension of policies to Deployments and ReplicaSets generated friction with ArgoCD during the GitOps pipeline integration phases, as the policies evaluated resources managed by the ArgoCD controller before they were fully compliant. This behaviour is documented in detail in Chapter 7, in the context of the complete pipeline integration.

6.6.1 Exclusion of infrastructure namespaces

During the initial implementation phases, validation policies generated violations on the cluster's own components. In particular, policy P1 (*disallow-root*) marked certain pods in the `kube-system` namespace as non-compliant, as they run as root due to functional requirements. In parallel, the first version of P3 (*restrict-image-registries*) did not include the `ghcr.io/*` pattern, which generated PolicyViolations on Kyverno's own controllers, whose images are distributed via `ghcr.io/kyverno/`.

The applied solution was twofold: `ghcr.io/*` was added to the list of authorised registries in P3, correcting the problem structurally; and the policies were revised to exclude infrastructure namespaces in cases where their application is not pertinent. This is a standard practice in Kyverno production deployments, where security policies must be designed with awareness of the system components that execute them. Figure 6.13 shows the final cluster state with all eight policies in Ready state.

```
student@control:~$ kubectl get clusterpolicies
NAME                                ADMISSION  BACKGROUND  READY  AGE  MESSAGE
add-default-labels                  true       true        True   20h  Ready
add-default-resources                true       true        True   20h  Ready
add-default-securitycontext          true       true        True   20h  Ready
add-readonly-rootfs                 true       true        True   20h  Ready
disallow-privilege-escalation        true       true        True   20h  Ready
disallow-root                       true       true        True   20h  Ready
generate-networkpolicy-metadata      true       true        True   20h  Ready
restrict-image-registries            true       true        True   20h  Ready
student@control:~$
```

Figure 6.13: All eight ClusterPolicies in Ready state after complete deployment

6.6.2 Integral auto-hardening test

The most representative test of the work consists of deploying a pod with deliberately minimal configuration and verifying that the complete set of mutation policies transforms it into a pod with a complete security profile. The initial manifest, shown in Figure 6.14, contains only the fields strictly necessary for operation: image, execution user and an `emptyDir` volume mounted at `/tmp` to satisfy policy P6's read-only filesystem requirement.

```
GNU nano 7.2 pod-seguro-completo.yaml *
apiVersion: v1
kind: Pod
metadata:
  name: pod-seguro-completo
  namespace: default
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
  containers:
    - name: nginx
      image: nginxinc/nginx-unprivileged
      volumeMounts:
        - name: tmp
          mountPath: /tmp
  volumes:
    - name: tmp
      emptyDir: {}
```

Figure 6.14: Minimal pod manifest used in the integral test

After deployment, pod inspection via `kubectl get pod pod-seguro-completo -o yaml` returns the complete object as persisted in etcd after passing through Kyverno's webhooks. Figures 6.15 and 6.16 show the result.


```

student@control:~$ kubectl apply -f pod-seguro-completo.yaml
pod/pod-seguro-completo created
student@control:~$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
pod-seguro-completo  1/1     Running   0           6s
test-mutate-resources 1/1     Running   0          12m
test-mutate-secontext 1/1     Running   0          26m
student@control:~$ kubectl get pod pod-seguro-completo -o yaml
apiVersion: v1
kind: Pod
metadata:
  annotations:
    cni.projectcalico.org/containerID: 13548428f118f11b8c357f51e09462dcc7dc931ebfa1cbebf96af38b55e94dc7
    cni.projectcalico.org/podIP: 172.16.171.112/32
    cni.projectcalico.org/podIPs: 172.16.171.112/32
    kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{},"name":"pod-seguro-completo","namespace":"default"},"spec":{"containers":[{"image":"nginxinc/nginx-unprivileged","name":"nginx","volumeMounts":[{"mountPath":"/tmp","name":"tmp"}]}],"securityContext":{"runAsNonRoot":true,"runAsUser":1000},"volumes":[{"emptyDir":{"name":"tmp"}}]}}
  creationTimestamp: "2026-04-14T11:40:53Z"
  generation: 1
  labels:
    security.tfm/hardened-by: kyverno
    security.tfm/policy-version: v1.0
  name: pod-seguro-completo
  namespace: default
  resourceVersion: "171272"
  uid: 19171f65-6aa5-4320-a838-7628514b3a53
spec:
  containers:
  - image: nginxinc/nginx-unprivileged
    imagePullPolicy: Always
    name: nginx
    resources:
      limits:
        cpu: 500m

```

Figure 6.15: Integral test (part 1): pod metadata and resources after admission

```
name: nginx
resources:
  limits:
    cpu: 500m
    memory: 256Mi
  requests:
    cpu: 100m
    memory: 128Mi
securityContext:
  allowPrivilegeEscalation: false
  capabilities:
    drop:
      - ALL
  readOnlyRootFilesystem: true
terminationMessagePath: /dev/termination-log
terminationMessagePolicy: File
volumeMounts:
- mountPath: /tmp
  name: tmp
- mountPath: /var/run/secrets/kubernetes.io/serviceaccount
  name: kube-api-access-v952w
  readOnly: true
dnsPolicy: ClusterFirst
enableServiceLinks: true
nodeName: worker
preemptionPolicy: PreemptLowerPriority
priority: 0
restartPolicy: Always
schedulerName: default-scheduler
securityContext:
  runAsNonRoot: true
  runAsUser: 1000
serviceAccount: default
serviceAccountName: default
terminationGracePeriodSeconds: 30
tolerations:
- effect: NoExecute
  key: node.kubernetes.io/not-ready
```

Figure 6.16: Integral test (part 2): complete security profile injected by Kyverno

The analysis of the persisted object confirms the presence of all fields injected by the four active mutation policies: `resources.requests` and `resources.limits` (P4); `allowPrivilegeEscalation: false` and `capabilities.drop: [ALL]` (P5); `readOnlyRootFilesystem: true` (P6); and labels `security.tfm/hardened-by: kyverno` and `security.tfm/policy-version: v1.0` (P7). Calico annotations additionally confirm that the pod has been assigned IP `172.16.171.112` in CIDR `172.16.0.0/16` and scheduled on the worker node.

This test constitutes the direct empirical evidence of the work's central thesis: a developer can define a pod with the minimum fields necessary for its function and obtain, in a completely transparent manner, a correctly secured pod that complies with the security profile defined by the organisation.

6.6.3 The mutation-validation order verified empirically

During testing, a behaviour was observed that empirically illustrates the webhook execution order described theoretically in Section 5.3.2. When attempting to deploy a pod with `allowPrivilegeEscalation: true` declared explicitly while both P5 and P2 were simultaneously active, the pod was created successfully despite P2 operating in Enforce mode.

The cause is precise: P5 (*mutating webhook*) executed before P2 (*validating webhook*) and overwrote the `true` value declared in the manifest with `false`, so that when P2 evaluated the resource, it already contained the compliant value. The `patchStrategicMerge` of P5 overwrites the field even when the developer has explicitly declared it with an insecure value.

This behaviour was verified by temporarily disabling P5 and applying the same pod: in the absence of mutation, P2 correctly rejected the request. Upon restoring P5, the pod was again admitted with the mutated value. This experiment empirically confirms that, in the adopted design, mutation has *absolute precedence* over validation, and that validation policies complement but do not replace mutation policies as the primary security mechanism.

6.7 Policy Reports: compliance auditing

Kyverno's Reports Controller automatically generates `PolicyReport` resources (namespace-scoped) and `ClusterPolicyReport` resources (cluster-scoped) that consolidate the results of evaluating all policies over all system resources. These reports constitute the native compliance audit mechanism and allow centrally answering the question: which cluster resources comply with the security policies?

Figure 6.17 shows the `ClusterPolicyReport` generated for the `test-networkpolicy` namespace: the `Namespace` type resource has passed the P8 evaluation, with result 1 PASS and 0 FAIL.

```
student@control:~$ kubectl get clusterpolicyreports
```

NAME	KIND	NAME	PASS	FAIL	WARN	ERROR	SKIP	AGE
59c08a41-a46a-469e-9503-e71e24500e73	Namespace	test-networkpolicy	1	0	0	0	0	124m

Figure 6.17: `ClusterPolicyReport`: evaluation of the generation policy on the test namespace

Figure 6.18 shows the `PolicyReport` objects for the default namespace, one per active pod. The three pods — `pod-seguro-completo`, `test-mutate-resources` and `test-mutate-secontext` — record 7 PASS and 0 FAIL.

The value 7 corresponds exactly to the number of policies that evaluate Pod type resources (P1 to P7). Policy P8 does not appear in these reports because its scope covers `Namespace` type resources, and its results are reflected in the `ClusterPolicyReport` objects of Figure 6.17.

```
student@control:~$ kubectl get policyreports -A
```

NAMESPACE	NAME	KIND	NAME	PASS	FAIL	WARN	ERROR	SKIP
AGE								
default	19171f65-6aa5-4320-a838-7628514b3a53	Pod	pod-seguro-completo	7	0	0	0	0
115m								
default	703ada7e-19ad-456d-91fc-e8d980b0db99	Pod	test-mutate-resources	7	0	0	0	0
127m								
default	749645f2-39c5-4b60-bef0-ecbf82301dbb	Pod	test-mutate-secontext	7	0	0	0	0

Figure 6.18: PolicyReports of the default namespace: 7 PASS per pod, 0 violations

These reports demonstrate that the implemented system not only acts at admission time, but also provides continuous visibility into the cluster's compliance state — a fundamental requirement in DevSecOps environments oriented towards auditing and regulatory compliance.

7 GitOps Integration with ArgoCD

This chapter documents the implementation of the Continuous Deployment layer of the DevSecOps environment, materialised through ArgoCD as the GitOps controller. Up to this point, Kyverno policies have been managed via direct `kubectl apply`, which implies that the cluster state and the state declared in the manifests are not continuously linked: any manual modification in the cluster can diverge from the desired state without any mechanism detecting or correcting it. This chapter implements the continuous synchronisation plane announced in Section 5.1, closing the DevSecOps architecture with the layer that connects the Git repository as the single source of truth with the real state of the cluster.

The chapter is organised into six sections. The first describes the GitOps repository design and the decision on its structure. The second documents ArgoCD's deployment in the cluster, including a relevant technical incident during installation. The third covers the creation of the `Application` that places Kyverno policies under declarative management, and the fourth documents a technical finding about the friction between defaults injected by Kyverno and ArgoCD's comparison engine. The fifth section collects three *drift protection* tests that empirically demonstrate the ArgoCD + Kyverno complementarity thesis of Chapter 5. The sixth section extends the GitOps architecture to a real workload, and the chapter concludes with a technical analysis of the interaction between Kyverno's *mutating webhooks* and ArgoCD's comparison engine, directly addressing the permanent *drift* risk warned about by the official documentation. The integration with the Continuous Integration plane, which will materialise the two-repository pattern described theoretically in Section 5.1, is addressed in Chapter 8 through the incorporation of the GitHub Actions pipeline.

7.1 GitOps repository design

The GitOps repository design requires a prior architectural decision: the number of repositories that will structure the system. Section 5.1 theoretically described the *two-repository pattern* — one for application source code and another for Kubernetes configuration — as the usual solution in production environments. Within the scope of this chapter, where the Continuous Integration pipeline has not yet been incorporated, a single repository is used —called `tfm-gitops`— which contains the Kyverno policies and demo application manifests, organised using folders that preserve the conceptual separation between planes. This structure will be expanded in Chapter 8 with the introduction of the second repository (`tfm-demo-app`), which will host the application source code and Dockerfile. The Continuous Integration pipeline will build

the image from that second repository and update the image tag in `tfm-gitops`, materialising the complete two-repository pattern announced in the architecture.

The final repository structure, visible in Figure 7.1, reflects three independent functional planes:

```
student@control:~/tfm-gitops$ tree ~/tfm-gitops
/home/student/tfm-gitops
├── applications
│   └── demo-app
├── argocd
├── policies
│   ├── 01-disallow-root.yaml
│   ├── 02-disallow-privilege-escalation.yaml
│   ├── 03-restrict-image-registries.yaml
│   ├── 04-add-default-resources.yaml
│   ├── 05-add-default-securitycontext.yaml
│   ├── 06-add-readonly-rootfs.yaml
│   ├── 07-add-default-labels.yaml
│   ├── 08-generate-networkpolicy-metadata.yaml
│   └── kustomization.yaml
└── README.md
```

Figure 7.1: Structure of the `tfm-gitops` repository with separation by functional planes

- `policies/`: contains the eight `ClusterPolicy` manifests (P1–P8) and the `kustomization.yaml` file that groups them.
- `applications/demo-app/`: contains the demonstrative workload manifests (Namespace, Deployment, Service) and their own `kustomization.yaml`.
- `argocd/`: contains `Application` type manifests that define how ArgoCD should manage each of the previous planes.

The `kustomization.yaml` file in the `policies/` folder plays a central role in the ArgoCD integration. Kustomize [22] is a native Kubernetes grouping mechanism that ArgoCD recognises automatically: when pointing an `Application` to a directory containing a `kustomization.yaml`, ArgoCD renders all listed resources and manages them as a logical unit. This avoids the need to individually reference each policy file in the ArgoCD configuration. Figure 7.2 shows the file content, listing the eight manifests in sequential order.

```
student@control:~/tfm-gitops$ cat policies/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- 01-disallow-root.yaml
- 02-disallow-privilege-escalation.yaml
- 03-restrict-image-registries.yaml
- 04-add-default-resources.yaml
- 05-add-default-securitycontext.yaml
- 06-add-readonly-rootfs.yaml
- 07-add-default-labels.yaml
- 08-generate-networkpolicy-metadata.yaml
student@control:~/tfm-gitops$
```

Figure 7.2: kustomization.yaml file in the policies/ folder with all eight policies

7.2 ArgoCD deployment

ArgoCD is installed in the cluster by applying the official manifests in the dedicated namespace `argocd`:

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/
↳ argo-cd/stable/manifests/install.yaml
```

7.2.1 Incident during installation: annotation size limit on CRDs

During the first execution of the above command, an error occurred that prevented the registration of the `ApplicationSet` CRD:

```
The ApplicationSet "applicationsets.argoproj.io" is invalid:
metadata.annotations: Too long: may not be more than 262144 bytes
```

The root cause of this error lies in the *client-side apply* mechanism used by default by `kubectl apply`: when applying a resource, `kubectl` serialises the complete manifest and stores it in the annotation `kubectl.kubernetes.io/last-applied-configuration` of the object. The OpenAPI schema of the `ApplicationSet` CRD exceeds the 256 KB limit imposed by Kubernetes for an individual annotation value, making it impossible to complete the operation through the standard mechanism.

The solution is to reinstall the manifests using *server-side apply* [23], which delegates the *merge* logic to the API Server instead of storing it in the client annotation:

```
kubectl apply -n argocd \
```

```
-f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/
↔ install.yaml \
--server-side --force-conflicts
```

The `--force-conflicts` flag is necessary because some manifest fields may have ended up in an inconsistent *field manager* state after the first partially completed attempt. With *server-side apply*, the `last-applied-configuration` field is not used and the CRD is correctly registered regardless of the size of its schema.

7.2.2 Deployment verification

After reinstallation, verification of pod state in the `argocd` namespace confirms that ArgoCD's seven components are active, as shown in Figure 7.3.

```
student@control:~/tfm-gitops$ kubectl get pods -n argocd
NAME                                READY   STATUS    RESTARTS   AGE
argocd-application-controller-0     1/1     Running   0           3m24s
argocd-applicationset-controller-5845f77959-blz8z  1/1     Running   1 (71s ago) 3m28s
argocd-dex-server-77c4b977cf-75kkw    1/1     Running   0           3m27s
argocd-notifications-controller-59f5f7587b-z4sqb  1/1     Running   0           3m27s
argocd-redis-fdb54b985-tqdaq         1/1     Running   0           3m26s
argocd-repo-server-5cb944c498-k8j2g    1/1     Running   0           3m26s
argocd-server-775bbd79f5-285lv       1/1     Running   0           3m25s
student@control:~/tfm-gitops$
```

Figure 7.3: ArgoCD's seven components in Running state in the `argocd` namespace

The seven pods correspond to the components described in Section 5.2.3: `application-controller` (continuous cluster state reconciliation), `applicationset-controller` (generation of `Application` objects from templates), `dex-server` (federated authentication), `notifications-controller` (alerts and notifications), `redis` (internal state cache), `repo-server` (Git repository cloning and rendering) and `server` (ArgoCD API and UI).

Verification of registered CRDs confirms that the three ArgoCD resource types are available in the cluster, as seen in Figure 7.4.

```
student@control:~/tfm-gitops$ kubectl get crds | grep argoproj
applications.argoproj.io                2026-04-23T19:37:24Z
applicationsets.argoproj.io            2026-04-25T17:25:43Z
appprojects.argoproj.io                2026-04-23T19:37:27Z
```

Figure 7.4: ArgoCD CRDs registered in the cluster: `applications`, `applicationsets` and `appprojects`

The ArgoCD dashboard (version 3.3.8) is accessible through port `8080` of the control node via

exposure of the `argocd-server` service. Figure 7.5 shows the initial interface state before registering any `Application`.

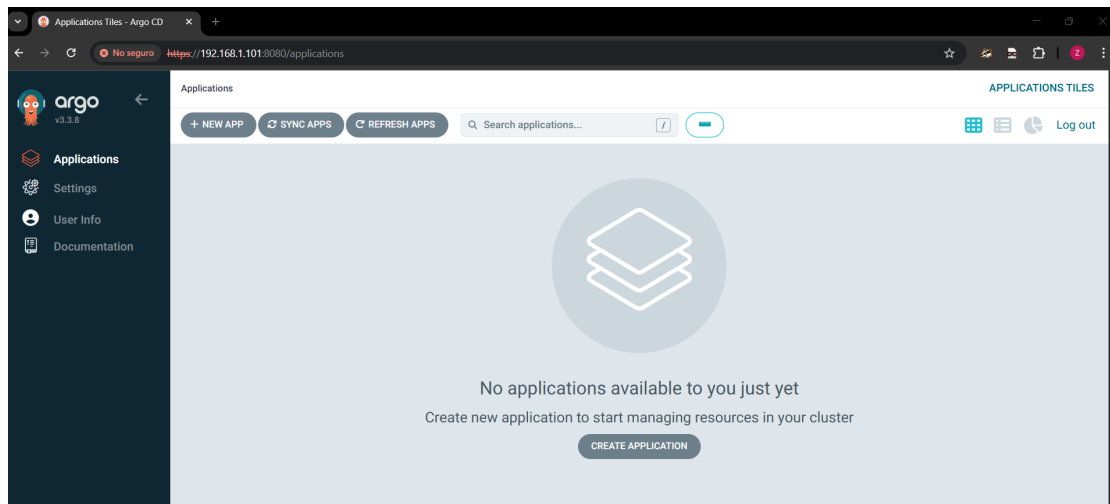


Figure 7.5: ArgoCD v3.3.8 interface accessible at `https://192.168.1.101:8080`

7.3 Application for Kyverno policies

An `Application` is ArgoCD's central CRD [18]: it represents the minimum unit of declarative management and encapsulates three elements —the Git source of the manifests, the destination in the cluster where they should be deployed and the synchronisation policy that defines how and when to synchronise. The manifest `app-policies.yaml`, shown in Figure 7.6, defines the `Application` that places the eight Kyverno policies under GitOps management.

```
GNU nano 7.2
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: kyverno-policies
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default
  source:
    repoURL: https://github.com/ezixd23/tfm-gitops
    targetRevision: main
    path: policies
  destination:
    server: https://kubernetes.default.svc
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
      - ServerSideApply=true
```

Figure 7.6: Manifest `app-policies.yaml`: Application managing Kyverno's ClusterPolicies

Each manifest field responds to a justified technical decision:

- `finalizers: resources-finalizer.argocd.argoproj.io`: activates *cascade delete*, so that when the ArgoCD Application is deleted, the synchronised resources in the cluster are also deleted. Guarantees that no orphan policies remain if the Application is removed.
- `automated.prune: true`: enables the automatic deletion of resources that exist in the cluster but have been removed from Git. Without this field, ArgoCD would detect the divergence but would not act, requiring manual intervention.
- `automated.selfHeal: true`: activates the automatic detection and reversal of manual modifications made directly on cluster resources. This field is the heart of the complementarity thesis with Kyverno formulated in Section 5.2.3: while Kyverno protects generated resources (NetworkPolicies), ArgoCD protects the very sources of that generation (ClusterPolicies).
- `syncOptions.ServerSideApply: true`: delegates resource *merge* to the API Server, avoiding the annotation size problem described in the previous section and being consistent with the installation of ArgoCD's own manifests.

- `syncOptions.CreateNamespace: true`: allows ArgoCD to create the destination namespace if it does not exist, although in this case the `kyverno` namespace was already created by Kyverno's prior installation.

7.3.1 Deployment and synchronisation process

Before applying the Application, the policies previously deployed via direct `kubectl` apply are removed from the cluster, so that the subsequent deployment unequivocally demonstrates that ArgoCD is the sole provisioning source:

```
kubectl delete clusterpolicies --all
kubectl apply -f ~/tfm-gitops/argocd/app-policies.yaml
```

Figure 7.7 shows the observed sequence of states: immediately after the manifest is applied, the Application `kyverno-policies` appears in `OutOfSync + Healthy` state, indicating that ArgoCD has detected the divergence between the repository (which contains the eight policies) and the cluster (which at that moment has none). In the next reconciliation cycle, ArgoCD synchronises the state and the Application transitions to `Synced + Healthy`.

```
student@control:~/tfm-gitops$ kubectl apply -f ~/tfm-gitops/argocd/app-policies.yaml
application.argoproj.io/kyverno-policies configured
student@control:~/tfm-gitops$ kubectl get application kyverno-policies -n argocd
NAME          SYNC STATUS  HEALTH STATUS
kyverno-policies OutOfSync    Healthy
student@control:~/tfm-gitops$ kubectl get application kyverno-policies -n argocd
NAME          SYNC STATUS  HEALTH STATUS
kyverno-policies Synced      Healthy
```

Figure 7.7: State transition `OutOfSync` → `Synced` after first deployment of `kyverno-policies`

Verification that the eight policies have been restored and are managed by ArgoCD is carried out in two steps. The first confirms the `Ready` state of the eight `ClusterPolicies`, as shown in Figure 7.8.

```
student@control:~/tfm-gitops$ kubectl get clusterpolicies
NAME                                ADMISSION  BACKGROUND  READY  AGE  MESSAGE
add-default-labels                  true       true        True   31m  Ready
add-default-resources                true       true        True   31m  Ready
add-default-securitycontext          true       true        True   31m  Ready
add-readonly-rootfs                 true       true        True   31m  Ready
disallow-privilege-escalation        true       true        True   31m  Ready
disallow-root                       true       true        True   31m  Ready
generate-networkpolicy-metadata      true       true        True   31m  Ready
restrict-image-registries             true       true        True   31m  Ready
```

Figure 7.8: The eight `ClusterPolicies` restored by ArgoCD in `Ready` state

The second step verifies that the *ownership* of each policy is established through ArgoCD's traceability annotation. Figure 7.9 shows that the `disallow-root` policy contains the annotation `argocd.argoproj.io/tracking-id: kyverno-policies:kyverno.io/ClusterPolicy:/disallow-root`, which unequivocally establishes that this resource is managed by the Application `kyverno-policies`.

```
student@control:~/tfm-gitops$ kubectl get clusterpolicy disallow-root -o yaml | grep -A 5 annotations
annotations:
  argocd.argoproj.io/tracking-id: kyverno-policies:kyverno.io/ClusterPolicy:/disallow-root
creationTimestamp: "2026-04-25T18:06:45Z"
generation: 1
name: disallow-root
resourceVersion: "266546"
```

Figure 7.9: ArgoCD *ownership* annotation in the `disallow-root` ClusterPolicy

ArgoCD's graphical interface, shown in Figure 7.10, presents the tree of synchronised resources: the Application `kyverno-policies` with its eight dependent ClusterPolicy objects, all in Synced state. The commit associated with the last successful synchronisation is visible in the status panel, confirming that the Git repository is the source of truth.

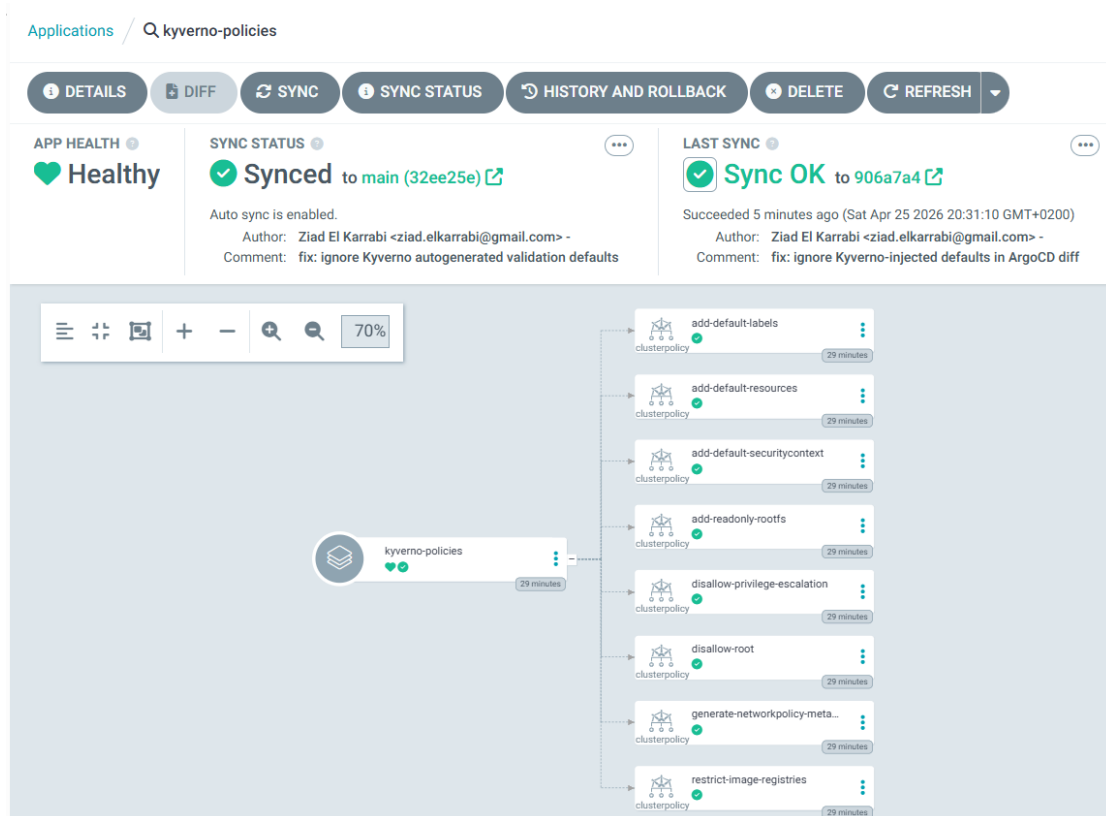
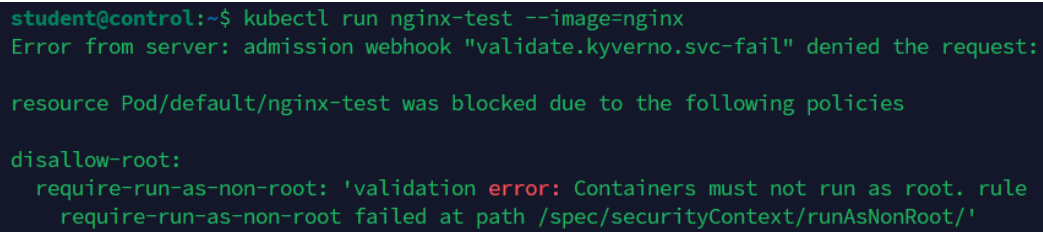


Figure 7.10: Tree view in ArgoCD: eight ClusterPolicies synchronised from the Git repository

The end-to-end functional test, shown in Figure 7.11, confirms that the policies managed by ArgoCD are functionally equivalent to those deployed manually: the attempt to create a

pod without `runAsNonRoot` is rejected by Kyverno's webhook with the same message as in Chapter 6 tests.



```
student@control:~$ kubectl run nginx-test --image=nginx
Error from server: admission webhook "validate.kyverno.svc-fail" denied the request:

resource Pod/default/nginx-test was blocked due to the following policies

disallow-root:
  require-run-as-non-root: 'validation error: Containers must not run as root. rule
    require-run-as-non-root failed at path /spec/securityContext/runAsNonRoot/'
```

Figure 7.11: End-to-end verification: insecure pod blocked by ArgoCD-managed policies

7.4 Managing false drift from Kyverno defaults

After the first successful deployment of the Application `kyverno-policies`, an unexpected behaviour was observed: the Application periodically reverted to `OutOfSync` state despite no changes having been made either in Git or in the cluster. Analysis of the ArgoCD diff revealed the root cause: Kyverno itself applies a *mutating webhook* on `ClusterPolicy` type resources at admission time, injecting default values into fields not specified in the Git manifests. Affected fields include `spec.validationFailureAction` (when not explicitly declared in some policy), `spec.rules[*].skipBackgroundRequests` and `spec.rules[*].allowExistingViolations`.

This situation is an example of what ArgoCD's documentation calls *false drift* [24]: the state in the cluster differs from the state in Git, but the difference is not the result of an unauthorised modification but of internal system behaviour. From ArgoCD's perspective, the resource in the cluster has additional fields or fields with different values than those declared in Git, which triggers the `selfHeal` mechanism unnecessarily, creating an unstable reconciliation loop.

7.4.1 Evaluation of resolution options

In this situation there are two resolution strategies:

1. **Add default values to the manifests in Git:** explicitly declare in each manifest the fields that Kyverno injects, so that the Git state and the cluster state match exactly.
2. **Configure `ignoreDifferences` in the Application:** instruct ArgoCD to ignore certain fields when performing the comparison, preserving the Git manifests in their minimal form.

The first option has a significant operational disadvantage: the default values injected by Kyverno are version-dependent. A Kyverno update that changes existing defaults or adds

new fields would require updating all Git manifests synchronously, creating an unnecessary coupling between the engine version and the policy manifests. This dependency violates the separation of responsibilities principle underlying the GitOps pattern: Git should contain the operator's *intent*, not an exact mirror of the system's internal state.

The second option, `ignoreDifferences` [24], allows expressing exactly this distinction: the ignored fields are internal management fields of the engine, not attributes of the policy declared by the operator. This strategy is adopted by adding the following block to the `app-policies.yaml` manifest:

```
ignoreDifferences:
- group: kyverno.io
  kind: ClusterPolicy
  jqPathExpressions:
  - .spec.rules[] | .skipBackgroundRequests
  - .spec.rules[] | .allowExistingViolations
  - .spec.rules[] | .validate? | .allowExistingViolations?
  - .spec.validationFailureAction
```

The `jqPathExpressions` use `jq` syntax to reference fields that should be excluded from comparison. The use of `?.` in `.validate?` avoids errors in *mutate* or *generate* rules where the `validate` field does not exist. After applying this configuration and re-synchronising, the `Application` remains stably in `Synced + Healthy` state, confirmed by the commit “fix: ignore Kyverno-injected defaults in ArgoCD diff” visible in the last synchronisation panel of Figure 7.10.

7.5 Drift protection tests

With the `Application` `kyverno-policies` stable, three tests are executed that empirically demonstrate the complementarity between ArgoCD and Kyverno as *drift protection* mechanisms at different architectural planes. Kyverno's mechanism, documented in Section 6.5 of the previous chapter, protects the resources *generated* by the policies (`NetworkPolicies`). ArgoCD's mechanism documented below protects the source policies themselves.

7.5.1 Self-healing upon manual modification

The first test simulates an internal actor — a cluster administrator with direct `kubectl` access — modifying a policy to weaken security controls. The scenario is realistic: in an environment without GitOps, this modification would be permanent and might go unnoticed. With ArgoCD and `selfHeal: true`, the cluster automatically reconciles with the Git declarative state.

The test modifies the `message` field of the `disallow-root` policy via `kubectl edit`, replacing the original text with the marker `DRIFT TEST - manually modified message`. Although modifying the error message does not affect validation logic, the change is sufficient for ArgoCD to detect the divergence. Figure 7.12 shows two consecutive queries: the first confirms that the modification was correctly applied (the mutated message appears in the three contexts generated by autogen); the second, executed moments later, shows that the message has returned to the original value `Containers must not run as root`, confirming that ArgoCD has reverted the change.

```
student@control:~/tfm-gitops$ kubectl edit clusterpolicy disallow-root
clusterpolicy.kyverno.io/disallow-root edited
student@control:~/tfm-gitops$ kubectl get clusterpolicy disallow-root -o yaml | grep "message:"
  message: DRIFT TEST - mensaje modificado manualmente
  message: DRIFT TEST - mensaje modificado manualmente
  message: DRIFT TEST - mensaje modificado manualmente
message: Ready
message: skip generating ValidatingAdmissionPolicy for non CEL rules.
student@control:~/tfm-gitops$ kubectl get clusterpolicy disallow-root -o yaml | grep "message:"
  message: Containers must not run as root
  message: Containers must not run as root
  message: Containers must not run as root
message: Ready
message: skip generating ValidatingAdmissionPolicy for non CEL rules.
```

Figure 7.12: Test 1: manual modification reverted by ArgoCD via `selfHeal`

In this experiment, the reversal occurred within seconds. Although the Git repository *polling* interval is configured by default to 3 minutes, ArgoCD maintains in parallel a continuous *watch* over synchronised resources via the Kubernetes API, allowing it to detect cluster changes almost in real time and initiate the reconciliation cycle immediately without waiting for the next Git *polling* cycle.

7.5.2 Policy deployment via Git commit

The second test demonstrates the GitOps flow in the downstream direction: Git as a provisioning mechanism without direct cluster access. A new policy, `disallow-latest-tag`, is created that rejects pods whose images are tagged with `:latest` — a practice that prevents deployment reproducibility by referring to an image whose content can change without notice. The policy is created directly in the repository's `policies/` directory, referenced in `kustomization.yaml` and uploaded via commit and push to the remote repository.

Figure 7.13 shows the complete process: file creation, `kustomization.yaml` modification, commit and push. The commit message, `feat: add disallow-latest-tag policy in kustomization.yaml`, documents the change's intent.

```

student@control:~/tfm-gitops/policies$ nano 09-disallow-latest-tag.yaml
student@control:~/tfm-gitops/policies$ cd ..
student@control:~/tfm-gitops$ git add policies/09-disallow-latest-tag.yaml
student@control:~/tfm-gitops$ git commit -m "feat: add disallow-latest-tag policy"
[main 4f54993] feat: add disallow-latest-tag policy
1 file changed, 20 insertions(+)
create mode 100644 policies/09-disallow-latest-tag.yaml
student@control:~/tfm-gitops$ git push origin main
Username for 'https://github.com': ezixd23
Password for 'https://ezixd23@github.com':
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 622 bytes | 207.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/ezixd23/tfm-gitops.git
32ee25e..4f54993 main -> main
student@control:~/tfm-gitops$ kubectl get clusterpolicy disallow-latest-tag
Error from server (NotFound): clusterpolicies.kyverno.io "disallow-latest-tag" not found
student@control:~/tfm-gitops$ nano policies/kustomization.yaml
student@control:~/tfm-gitops$ git add policies/kustomization.yaml
student@control:~/tfm-gitops$ git commit -m "feat: add disallow-latest-tag policy in kustomization.yaml"
[main lead55f] feat: add disallow-latest-tag policy in kustomization.yaml
1 file changed, 1 insertion(+)
student@control:~/tfm-gitops$ git push origin main
Username for 'https://github.com': ezixd23
Password for 'https://ezixd23@github.com':
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 414 bytes | 414.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/ezixd23/tfm-gitops.git
4f54993..lead55f main -> main
student@control:~/tfm-gitops$

```

Figure 7.13: Test 2: new policy created via Git commit

Figure 7.14 confirms that the `disallow-latest-tag` policy has been deployed in the cluster by ArgoCD without any direct intervention, with an age of 2 minutes and 42 seconds compared to the other policies that have been active for over an hour. The `BACKGROUND: false` field reflects that this policy, due to its exclusion condition-based design, does not generate retroactive evaluations on existing pods.

```

student@control:~/tfm-gitops$ kubectl get clusterpolicy disallow-latest-tag

```

NAME	ADMISSION	BACKGROUND	READY	AGE	MESSAGE
disallow-latest-tag	true	false	True	2m42s	Ready

```

student@control:~/tfm-gitops$

```

Figure 7.14: Test 2: `disallow-latest-tag` automatically deployed by ArgoCD (2m42s age)

ArgoCD's interface, shown in Figure 7.15, reflects the updated tree with nine `ClusterPolicies` — the new policy appears as an additional node in the graph with a timestamp of 2 minutes ago, while the rest has been synchronised for over an hour.

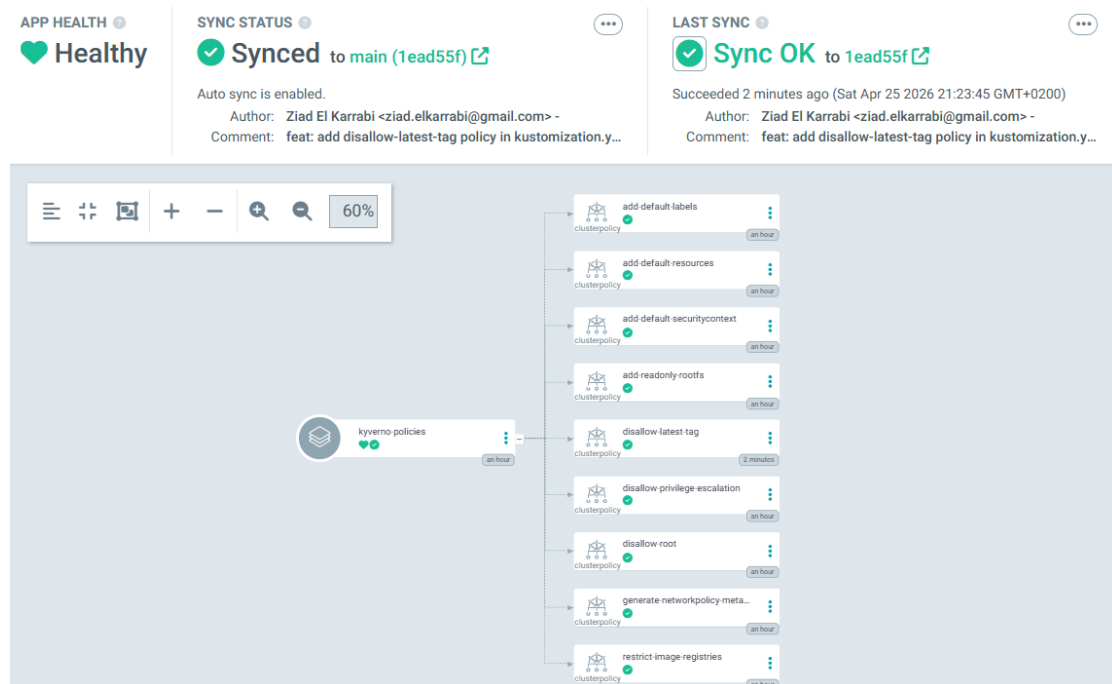


Figure 7.15: Test 2: ArgoCD tree updated with the new disallow-latest-tag policy

7.5.3 Protection against deletion

The third test verifies that ArgoCD automatically restores a policy deleted directly from the cluster. This scenario complements the Kyverno *drift protection* test documented in Section 6.5, where it was demonstrated that the Background Controller restores the block-cloud-metadata NetworkPolicy generated by P8. In this case, the deletion affects the source policy that generates it.

The disallow-root policy is deleted via `kubectl delete clusterpolicy disallow-root`. Figure 7.16 shows the sequence: the deletion is confirmed with the message `clusterpolicy.kyverno.io "disallow-root" deleted`; the immediate subsequent query returns an empty result. However, 48 seconds later, a new query shows the policy again in Ready state, with an age of 48 seconds compared to the minutes of the others, confirming that ArgoCD has recreated it.

```
student@control:~/tfm-gitops$ kubectl delete clusterpolicy disallow-root
clusterpolicy.kyverno.io "disallow-root" deleted
student@control:~/tfm-gitops$ kubectl get clusterpolicy disallow-root
NAME            ADMISSION  BACKGROUND  READY  AGE    MESSAGE
disallow-root   true      true        True   48s    Ready
student@control:~/tfm-gitops$
```

Figure 7.16: Test 3: disallow-root policy restored by ArgoCD 48 s after manual deletion

The combination of the three tests demonstrates that the system operates with two orthogonal protection layers: ArgoCD guarantees that policies defined in Git always exist and are not modified; Kyverno guarantees that resources generated by those policies (NetworkPolicies) cannot be deleted or modified either. The removal of either protection layer does not completely compromise system security, but does reduce its resilience surface.

7.6 Application for the demo workload

The GitOps architecture implemented up to this point manages only the platform policy plane. To complete the two-plane architecture described in Section 5.1 and demonstrate that the system works in an integrated manner for real workloads, a second **Application** is implemented that deploys a demo application managed declaratively from the same GitOps repository.

7.6.1 Demo application design

The chosen base image is `nginxinc/nginx-unprivileged:1.27-alpine`, available on `docker.io` under the `nginxinc/` prefix that satisfies policy P3 (`restrict-image-registries`). The *unprivileged* variant of the official nginx image is specifically designed to run without root privileges: the nginx process listens on port 8080 instead of port 80 (which would require `CAP_NET_BIND_SERVICE`) and runs with UID 101, being compatible with policy P1 (`disallow-root`) without additional configuration.

The demo application manifests are three:

Namespace (`namespace.yaml`): declares the `demo-app` namespace with identification labels. The label `purpose: tfm-demo` is relevant because its existence activates policy P8 (`generate-networkpolicy-metadata`), which will automatically generate the IMDS blocking NetworkPolicy as soon as the namespace is created.

Deployment (`deployment.yaml`), whose content is shown in Figure 7.17: specifies 2 replicas of the `nginx-unprivileged` image, the pod-level `securityContext` with `runAsNonRoot: true` and `runAsUser: 101`, and three `emptyDir` volume mounts — `/tmp`, `/var/cache/nginx` and `/var/run`. The inclusion of these three volumes directly responds to policy P6 (`add-readonly-rootfs`), which forces the root filesystem to read-only mode: nginx requires writing PID files to `/var/run`, cache files to `/var/cache/nginx` and temporary files to `/tmp`, and without the corresponding volumes the process would fail to start. This configuration materialises the lesson documented in Section 6.4.3: the policy is not weakened, but rather forces the developer to be explicit about their application's write requirements.

```
GNU nano 7.2
apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo-app
  namespace: demo-app
  labels:
    app: demo-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: demo-app
  template:
    metadata:
      labels:
        app: demo-app
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 101
      containers:
        - name: nginx
          image: nginxinc/nginx-unprivileged:1.27-alpine
          ports:
            - containerPort: 8080
              name: http
          volumeMounts:
            - name: tmp
              mountPath: /tmp
            - name: nginx-cache
              mountPath: /var/cache/nginx
            - name: nginx-run
              mountPath: /var/run
      volumes:
        - name: tmp
          emptyDir: {}
        - name: nginx-cache
```

Figure 7.17: Demo application deployment.yaml manifest with three emptyDir volumes

Service (service.yaml): exposes the application via a ClusterIP that maps port 80 to the container's port 8080, with an additional NodePort configured at 8888 for laboratory access.

7.6.2 Application manifest and deployment

The manifest app-demo.yaml, shown in Figure 7.18, defines the ArgoCD Application for the workload. There is a deliberate design difference compared to app-policies.yaml: the selfHeal field is configured to false.

```
GNU nano 7.2
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: demo-app
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default
  source:
    repoURL: https://github.com/ezixd23/tfm-gitops
    targetRevision: main
    path: applications/demo-app
  destination:
    server: https://kubernetes.default.svc
    namespace: demo-app
  syncPolicy:
    automated:
      prune: true
      selfHeal: false
    syncOptions:
      - CreateNamespace=true
      - ServerSideApply=true
```

Figure 7.18: Manifest `app-demo.yaml`: Application for the workload with `selfHeal: false`

This decision reflects an important operational distinction between the two managed planes. Security policies are platform assets whose integrity is critical: any deviation from their declarative state in Git constitutes a potential security posture degradation and must be automatically reverted. Workload resources, on the other hand, may be temporarily modified by the operations team for diagnostics, by auto-scaling scripts or by other cluster controllers without representing a security problem. The `selfHeal: false` delegates to the team the decision of when to reconcile, maintaining declarative control (Git as source of truth) but without the automatic reversal that can interfere with legitimate operations.

After applying the manifest, ArgoCD detects the divergence between the repository and the cluster (which does not yet contain any resources from the `demo-app` namespace) and proceeds with synchronisation. Figure 7.19 shows the resulting resource tree: the Application `demo-app` manages the Namespace, Service and Deployment, which in turn generates a ReplicaSet with two active Pods.

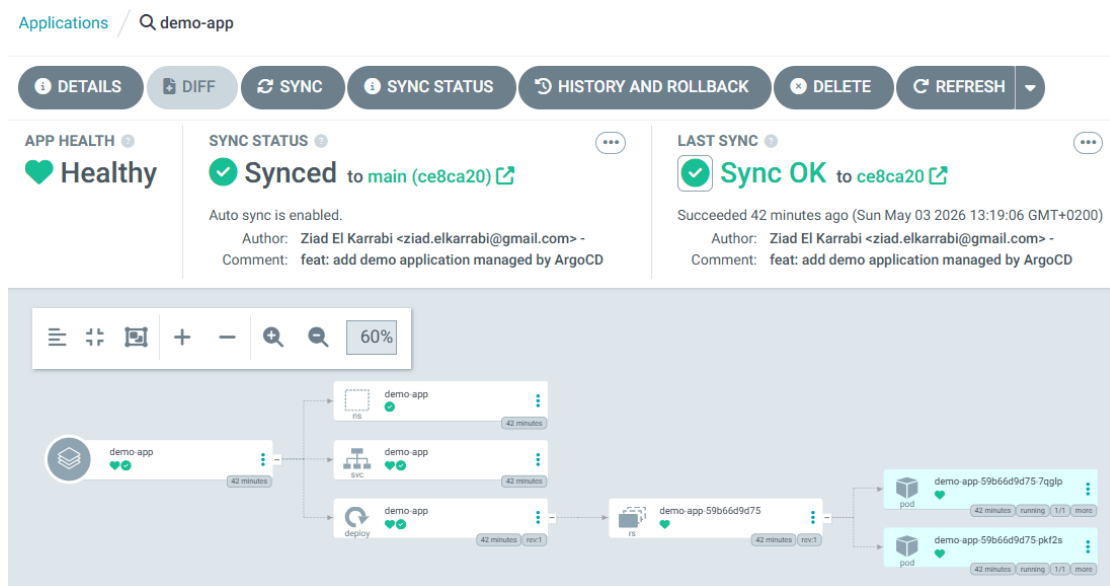


Figure 7.19: Resource tree of the demo-app Application: Namespace, Service, Deployment and two Pods

7.6.3 Verification of Kyverno mutations

Figure 7.20 shows the integrated verification of the effects of mutation policies on the demo application pods.

```
student@control:~/tfm-gitops$ kubectl get networkpolicies -n demo-app
NAME                                POD-SELECTOR  AGE
block-cloud-metadata                <none>        44m
student@control:~/tfm-gitops$ kubectl get pods -n demo-app -o yaml | grep -A 6 "resources:"
resources:
  limits:
    cpu: 500m
    memory: 256Mi
  requests:
    cpu: 100m
    memory: 128Mi
--
resources:
  limits:
    cpu: 500m
    memory: 256Mi
  requests:
    cpu: 100m
    memory: 128Mi
--
resources:
  limits:
    cpu: 500m
    memory: 256Mi
  requests:
    cpu: 100m
    memory: 128Mi
--
resources:
  limits:
    cpu: 500m
    memory: 256Mi
  requests:
    cpu: 100m
    memory: 128Mi
student@control:~/tfm-gitops$ kubectl get pods -n demo-app --show-labels
NAME                                READY  STATUS   RESTARTS  AGE    LABELS
demo-app-59b66d9d75-7qglp          1/1    Running  0          44m    app=demo-app,pod-template-hash=59b66d9d75,security.tfm/hardened-by=kyverno,security.tfm/policy-version=v1.0
demo-app-59b66d9d75-pk72s          1/1    Running  0          44m    app=demo-app,pod-template-hash=59b66d9d75,security.tfm/hardened-by=kyverno,security.tfm/policy-version=v1.0
student@control:~/tfm-gitops$
```

Figure 7.20: Verification of mutations P4, P7 and P8 on the demo application pods

The query `kubectl get networkpolicies -n demo-app` confirms that the NetworkPolicy `block-cloud-metadata` has been automatically generated by P8 in the demo-app

namespace at the moment of its creation, without any manual intervention. The NetworkPolicy has existed for 44 minutes, an identical period to the rest of the namespace resources, confirming that generation occurred immediately.

The query of the following command:

```
kubectl get pods -n demo-app -o yaml | grep -A 6 "resources:"
```

shows that all containers in the ReplicaSet pods have injected `limits` (cpu: 500m, memory: 256Mi) and `requests` (cpu: 100m, memory: 128Mi) fields by policy P4, despite the Deployment declared in Git specifying no `resources` field. The count of four `resources` blocks is due to the `grep` command capturing all occurrences of the `resources:` pattern in the pod's YAML manifest—both in `spec.containers` (the desired specification) and in `status.containerStatuses` (the actual state reported by the kubelet)—, repeated for the two active pods in the ReplicaSet.

The query `kubectl get pods -n demo-app --show-labels` confirms the presence of the labels `security.tfm/hardened-by: kyverno` and `security.tfm/policy-version: v1.0` injected by P7 on both pods, alongside the Deployment's own labels (`app: demo-app`, `pod-template-hash`). The HTTP verification, shown in Figure 7.21, confirms that the application is functionally correct despite the complete set of applied security restrictions: `nginx` responds correctly at `192.168.1.101:8888`.

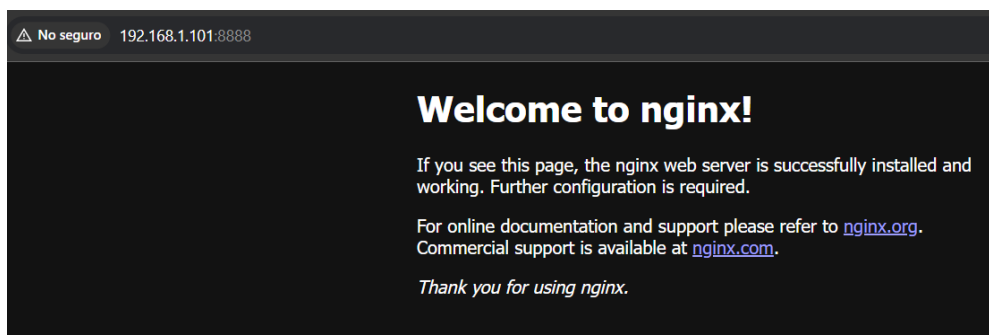


Figure 7.21: Demo application accessible and functional at `192.168.1.101:8888` with all policies active

Figure 7.22 shows the final ArgoCD panel state with both `Application` objects—`demo-app` and `kyverno-policies`—both in `Healthy + Synced` state, materialising the two-plane GitOps architecture described in Chapter 5.

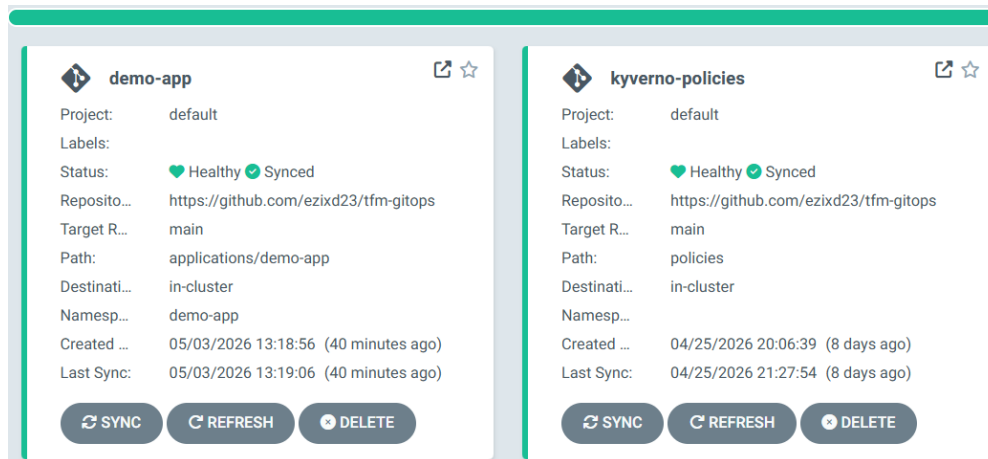


Figure 7.22: Final ArgoCD panel state: two Applications in Healthy and Synced state

7.7 Analysis: interaction between mutating webhooks and ArgoCD

ArgoCD's official documentation explicitly warns that the presence of *mutating admission controllers* in the cluster can generate permanent *drift* [24]: if a webhook modifies a resource after ArgoCD has synchronised it, the state in the cluster will differ from the state in Git indefinitely, chronically maintaining the Application in `OutOfSync` state. The prescribed solution is the `ignoreDifferences` or `ServerSideApply` options. Section 7.4 already documented the application of this solution for the case of defaults injected by Kyverno on the `ClusterPolicies` themselves. This section analyses whether the same problem manifests for the mutations Kyverno applies to the demo application pods.

7.7.1 Observed behaviour: absence of drift in the demo Application

The Application `demo-app` remains stably in `Synced + Healthy` state, as confirmed by Figure 7.23, despite its pods being mutated by the four policies P4–P7. This result, which might seem counterintuitive given the documentation warning, has a precise technical explanation.

```

student@control:~$ kubectl get applications -n argocd
kubectl get application demo-app -n argocd -o jsonpath='{.status.sync.status}'
echo ""
kubectl get application demo-app -n argocd -o jsonpath='{.status.health.status}'
echo ""
NAME           SYNC STATUS  HEALTH STATUS
demo-app       Synced      Healthy
kyverno-policies Synced      Healthy
Synced
Healthy
student@control:~$

```

Figure 7.23: Both Applications in Synced and Healthy state: pod mutation does not generate drift in ArgoCD

ArgoCD compares resources at the same hierarchical level at which they are declared in Git. The repository contains a resource of type Deployment, not Pod resources directly. Kyverno's mutations —P4 (resources), P5 (securityContext), P6 (readOnlyRootFilesystem) and P7 (labels)— are applied to the Pod at the time of its creation by the ReplicaSet, not to the `spec.template` field of the Deployment. ArgoCD compares the Deployment declared in Git with the Deployment existing in the cluster, and these match. The Pod resources generated as child resources are not directly managed by ArgoCD and do not form part of the comparison.

The empirical verification of this behaviour, documented in Figure 7.24, compares the content of the Deployment and a Pod in the cluster by searching for relevant fields:

```

student@control:~$ kubectl get deployment demo-app -n demo-app -o yaml > /tmp/deployment-cluster.yaml
student@control:~$ kubectl get pod -n demo-app -l app=demo-app -o yaml > /tmp/pod-cluster.yaml
student@control:~$ grep -E "resources:|securityContext:|hardened-by" /tmp/deployment-cluster.yaml
security.tfm/hardened-by: kyverno
resources:
securityContext:
securityContext:
student@control:~$ grep -E "resources:|securityContext:|hardened-by" /tmp/pod-cluster.yaml
security.tfm/hardened-by: kyverno
resources:
securityContext:
securityContext:
resources:
security.tfm/hardened-by: kyverno
resources:
securityContext:
securityContext:
resources:
student@control:~$

```

Figure 7.24: Deployment vs. Pod comparison: mutations exist in the Pod but not in the `spec.template` of the Deployment

The search on `/tmp/deployment-cluster.yaml` returns `resources:` with empty value (the field exists in the object but without assigned values in the template) and `securityContext:`

only with `runAsNonRoot: true` and `runAsUser: 101`, which correspond to the values explicitly declared in the Git manifest. The search on `/tmp/pod-cluster.yaml` returns `resources:` with values injected by P4 (`limits` and `requests`), multiple `securityContext:` entries with values injected by P5 and P6, and the presence of the label `security.tfm/hardened-by: kyverno` injected by P7. The injected fields exist in the Pod but not in the Deployment, and ArgoCD only compares Deployments.

This observation is related to the autogen mechanism described in Section 6.2.1. The rules `autogen-add-resources`, `autogen-add-default-securitycontext` and similar, visible in Figure 7.25, generate additional rules that apply to the `spec.template.spec` of higher-level controllers, not only to the resulting Pods.

```
student@control:~$ for policy in add-default-resources add-default-securitycontext add-readonly-rootfs add-default-labels; do echo "=== $policy ==="; kubectl get clusterpolicy $policy
yml | grep -A 50 "autogen:"; echo ""; done
== add-default-resources ==
autogen:
  rules:
  - match:
    - resources:
        any:
        - kinds:
            - DaemonSet
            - Deployment
            - Job
            - ReplicaSet
            - ReplicationController
            - StatefulSet
          resources: {}
      mutate:
        patchStrategicMerge:
          spec:
            template:
              spec:
                containers:
                - (name): '*'
                  resources:
                    limits:
                      cpu: 500m
                      memory: 256Mi
                    requests:
                      cpu: 100m
                      memory: 128Mi
          name: autogen-add-resources
          skipBackgroundRequests: true
  - match:
    - resources:
        any:
        - kinds:
            - CronJob
          resources: {}
      mutate:
        patchStrategicMerge:
          spec:
```

Figure 7.25: Autogen rules for policy P4: cover Deployment, StatefulSet, Job, ReplicaSet via `spec.template.spec`

However, empirical comparison demonstrates that the `spec.template` of the Deployment in the cluster does not contain the injected fields. This indicates that in the version of Kyverno used (v1.13) with `patchStrategicMerge`, the effective mutation occurs only on the Pod object at the time of its creation by the ReplicaSet, not on the `spec.template` of the Deployment at admission time. This behaviour is consistent with the principle of minimum interference with external declarative systems such as ArgoCD.

7.7.2 Verification of the case where drift does occur

To confirm that the described mechanism is correct and not simply an absence of mutation, an experimental policy —`experimental-deployment-labels`— is designed that explicitly mutates Deployment type resources by adding a label `experimental.tfm/`

deployment-mutated: "true" directly to the Deployment object, as shown in Figure 7.26.

```
GNU nano 7.2
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: experimental-deployment-labels
  annotations:
    policies.kyverno.io/title: Experimental - Deployment-level mutation
    policies.kyverno.io/description: >-
      Política experimental utilizada en el TFM para demostrar la fricción
      entre ArgoCD y los mutating webhooks cuando la mutación se aplica
      a nivel de Deployment. No forma parte del set de políticas principal.
spec:
  rules:
    - name: add-experimental-label
      match:
        any:
          - resources:
              kinds:
                - Deployment
      mutate:
        patchStrategicMerge:
          spec:
            template:
              metadata:
                labels:
                  experimental.tfm/deployment-mutated: "true"
```

Figure 7.26: Experimental policy that directly mutates Deployment type resources

Figure 7.27 shows the result: immediately after applying this policy in the cluster, the Application demo-app transitions to OutOfSync state while kyverno-policies remains Synced. The Deployment in the cluster now has the additional label not declared in Git, and ArgoCD correctly detects the divergence.

```
student@control:~/tfm-gitops/applications/demo-app$ kubectl get applications -n argocd
NAME          SYNC STATUS  HEALTH STATUS
demo-app      OutOfSync    Healthy
kyverno-policies Synced       Healthy
student@control:~/tfm-gitops/applications/demo-app$
```

Figure 7.27: Real drift case test: demo-app transitions to OutOfSync when directly mutating the Deployment

This experiment simultaneously confirms two things: first, that ArgoCD's drift detection mechanism works correctly when mutation affects the level of the managed resource; second, that the main set's mutation policies (P4–P7) do not produce this effect because they operate at Pod level and not at Deployment level, which constitutes the correct design for their coexistence with GitOps systems.

7.7.3 Conclusion and resolution pattern

Empirical analysis allows formulating a precise technical conclusion: in the implemented setup, the friction between Kyverno's *mutating webhooks* and ArgoCD does not manifest for the main set's mutation policies because these operate at a different hierarchical level than the resources managed by ArgoCD. The official documentation warning is correct in general, but not applicable to this specific design.

In scenarios where policies directly mutated resources of the same type as those managed in Git—for example, `spec.replicas` mutations in Deployments or addition of labels at the Deployment level—it would be necessary to configure `ignoreDifferences` in the corresponding `Application` for the affected fields. The resolution pattern already documented in Section 7.4—use of `jqPathExpressions` to exclude fields modified by the system—is directly applicable to this case, without requiring changes to the Git manifests or Kyverno policies.

Bibliography

- [1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*. McGraw-Hill, 2019.
- [2] G. Kim, J. Humble, P. Debois, and J. Willis, "The devops handbook," 2016.
- [3] T. Sharma et al., "Devsecops: A multivocal literature review," 2020. [Online]. Available: <https://arxiv.org/abs/2007.12561>
- [4] A. Behl and K. Behl, "Devsecops: Integrating security into devops," 2017.
- [5] A. Rahman and L. Williams, "Devsecops: Integrating security into continuous delivery," 2019.
- [6] V. Mohan and L. Ben Othmane, "Towards devsecops: Integrating security into devops," 2016. [Online]. Available: <https://ieeexplore.ieee.org/document/7488962>
- [7] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, omega, and kubernetes," *Communications of the ACM*, 2016. [Online]. Available: <https://dl.acm.org/doi/10.1145/2934556.2934564>
- [8] The Kubernetes Authors, *Kubernetes documentation*, 2024. [Online]. Available: <https://kubernetes.io/docs/home/>
- [9] M. Hüttermann, *DevOps for Developers*. 2022.
- [10] Kyverno Project, *Kyverno documentation*, 2024. [Online]. Available: <https://kyverno.io/>
- [11] OPA Project, *Open policy agent documentation*, 2024. [Online]. Available: <https://www.openpolicyagent.org/>
- [12] Weaveworks, *Gitops explained*, 2021. [Online]. Available: <https://www.weave.works/technologies/gitops/>
- [13] GitHub, *Github actions documentation*, 2024. [Online]. Available: <https://docs.github.com/en/actions>
- [14] Aqua Security, *Trivy documentation*, 2024. [Online]. Available: <https://aquasecurity.github.io/trivy/>
- [15] Sysdig, *Sysdig secure*, 2024. [Online]. Available: <https://sysdig.com/products/secure/>
- [16] Tigera, *Calico documentation*, 2024. [Online]. Available: <https://docs.tigera.io/calico/latest/about/>

- [17] Socket Security, *Teampcp partners with vect targeting oss supply chains*, Accessed: April 2026, 2026. [Online]. Available: <https://socket.dev/blog/teampcp-partners-with-vect-targeting-oss-supply-chains>
- [18] Argo Project, *Argocd documentation*, 2024. [Online]. Available: <https://argo-cd.readthedocs.io/>
- [19] Linux Kernel Developers, *no_new_privs — The Linux Kernel documentation*, 2024. [Online]. Available: https://www.kernel.org/doc/html/latest/userspace-api/no_new_privs.html
- [20] The Kubernetes Authors, *Configure Quality of Service for Pods*, 2024. [Online]. Available: <https://kubernetes.io/docs/tasks/configure-pod-container/quality-service-pod/>
- [21] U.S. Department of Justice, *Capital One Data Breach — U.S. Department of Justice*, Caso documentado de explotación del Instance Metadata Service (IMDS) mediante SSRF, 2019. [Online]. Available: <https://www.justice.gov/usao-wdwa/pr/seattle-tech-worker-charged-data-theft-involving-capital-one>
- [22] The Kubernetes Authors, *Kustomize — Kubernetes-native configuration management*, 2024. [Online]. Available: <https://kustomize.io/>
- [23] The Kubernetes Authors, *Server-Side Apply*, 2024. [Online]. Available: <https://kubernetes.io/docs/reference/using-api/server-side-apply/>
- [24] Argo Project, *ArgoCD Diffing Customization*, 2024. [Online]. Available: <https://argo-cd.readthedocs.io/en/stable/user-guide/diffing/>

